Unit 6
Timing and Critical Path Analysis

EL-GY 9463: INTRODUCTION TO HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

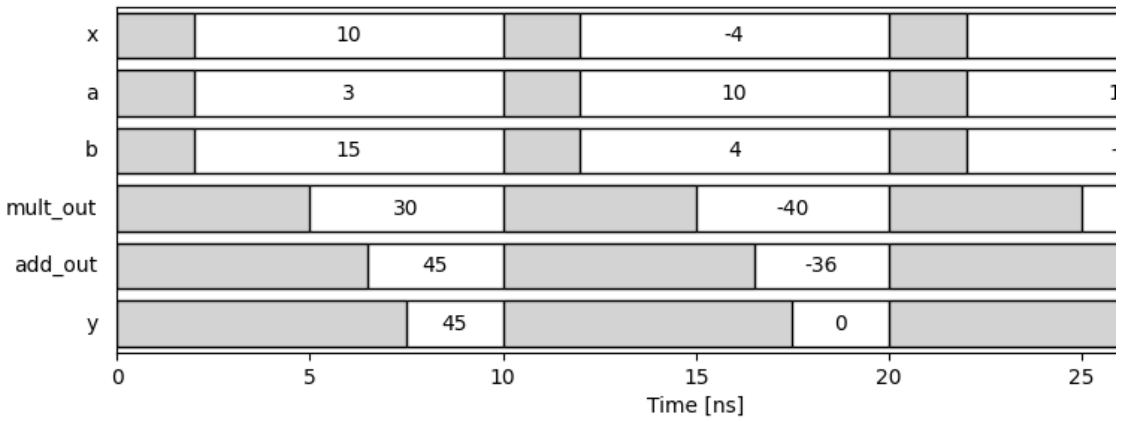
- ❑ Define undefined and stable periods for logical signals
- ❑ Compute the maximum propagation delay of combination logics
  - Series circuits, conditional, multi-path
- ❑ Perform static timing analysis of sequential logic
  - Determine the critical path of a circuit
  - Determine maximum frequency and detect timing violations
- ❑ Draw timing diagrams, both real and cycle-accurate
- ❑ Describe timing via value-change points
- ❑ Extract VCD files from Vitis and display timing information in python

Outline

- Static Timing Analysis in Combinational Logic
- Static Timing Analysis in Sequential Logic
- Sequential Logic Timing Diagrams
- VCD Files and Timing Diagrams from Vitis

Static Timing Analysis

- Timing analysis: Estimate delays through circuits
- Visualize with a timing diagram
- Critical for estimating:
  - Clock speed, latency, throughput, …
- Static timing analysis:
  - Focus on maximum propagation delay
  - Timing that can be guaranteed for all inputs
  - Not data dependent
- This analysis is conservative, but provides hard guarantees
- Default analysis in industry

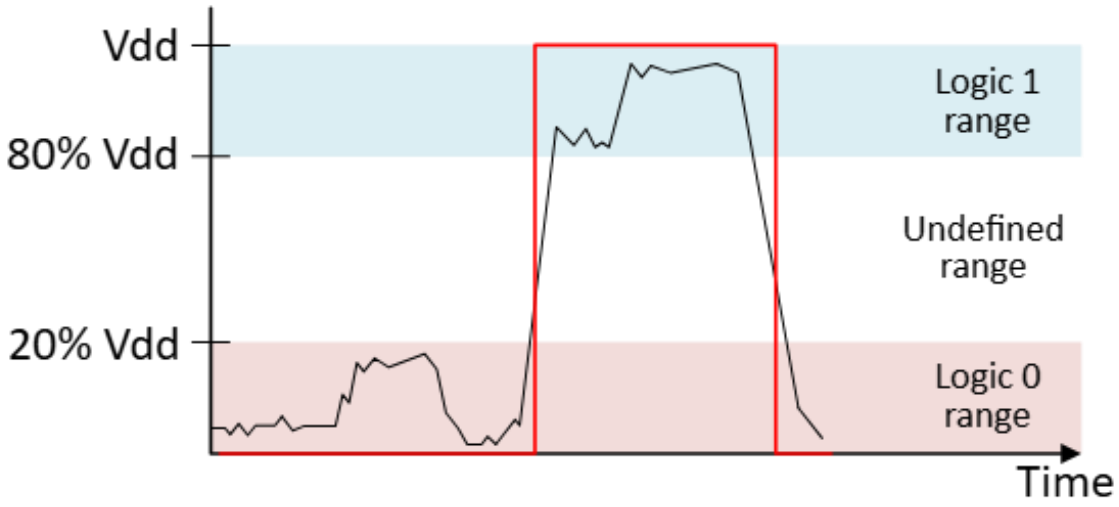


Implementing Signal Physically

- To understand timing, we need to look at physical implementation of signals

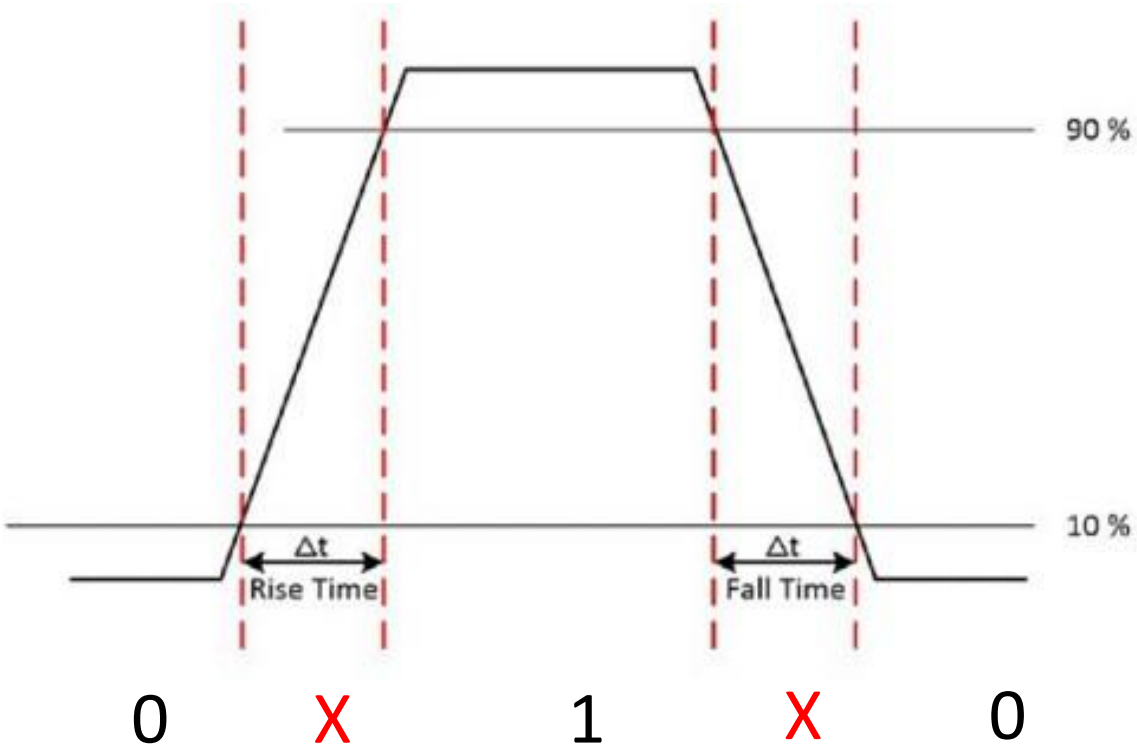
- Consider a binary signal from Unit 1
- Two logical values: 0 or 1

- What happens physically?
  - Typically, a voltage on a wire
  - Takes a continuous value
  - Obtain logical value by thresholding

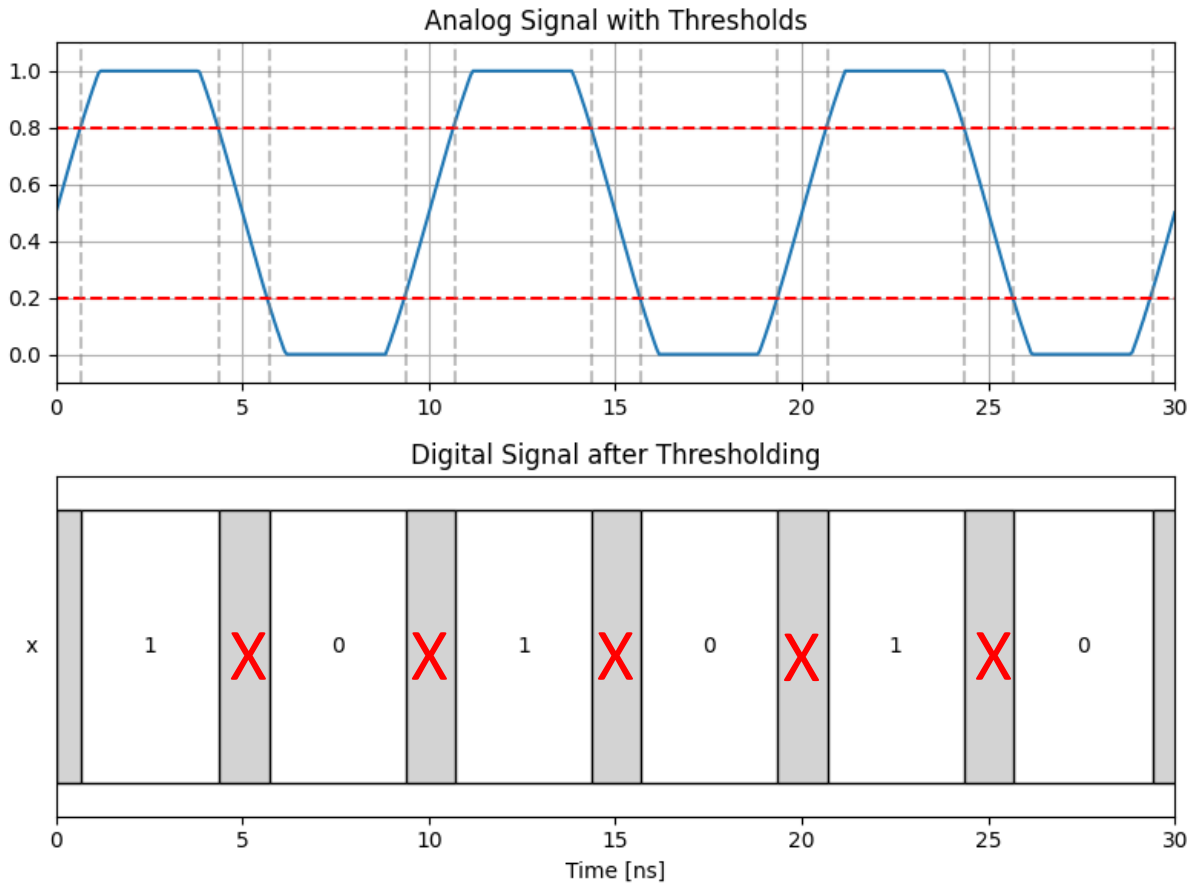


Undefined Logic Value: ‘X’ or ‘?’

- Signals take time to transition
  - They are outputs of physical circuits
  - Gates, computational units, …
- Creates an undefined period
  - Also called indeterminate, unknown, …
- Usually denoted as X
  - We may use ? to avoid confusion with variable
- So binary logic signals have three values:
  - 0, 1 and X
  - Will discuss a fourth value Z later



Undefined Regions Illustrated

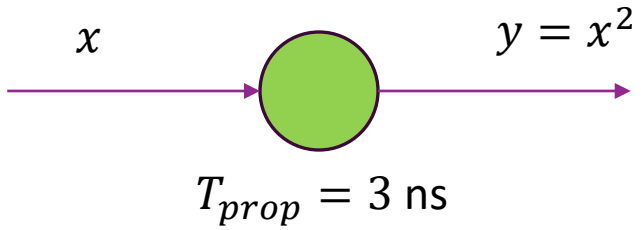


- Analog physical signal
  - Typically, a voltage
  - Continuous valued
  - Has finite transition periods

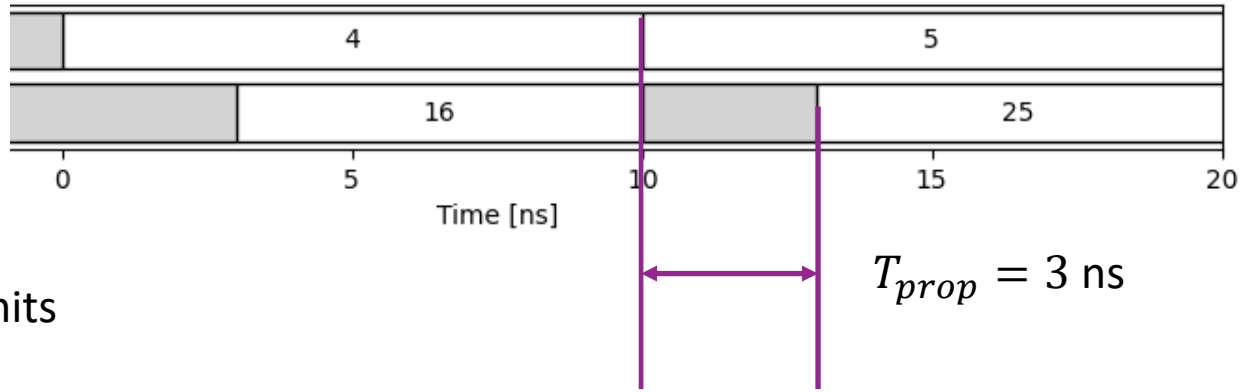
- Digital logic signal
  - Three values 0, 1 and X
  - X value shown in gray

Stable Values and Propagation Delays

- Stable value: Eventual value after a transition
  - Ex: Suppose $y = x^2$ and x transitions from 4 to 5:
  - Output y is eventually $y = 5^2 = 25$
  - Stable output value is $y = 25$

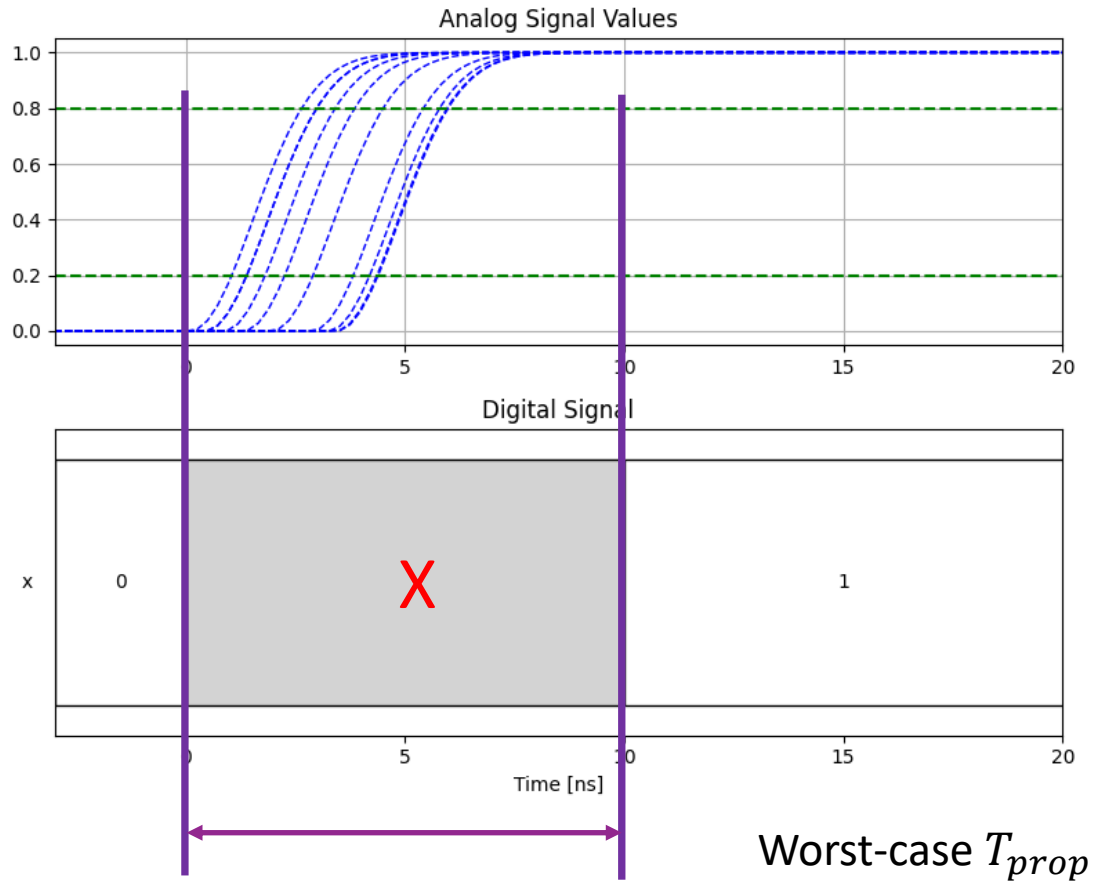


- Propagation delay:
  - Time for the output to transition



- Two components:
  - Logic delay: Time for the logic gates to settle
  - Routing delay: Time to pass through wire
  - May be related to capacitance, current source limits

Maximum Propagation Delay



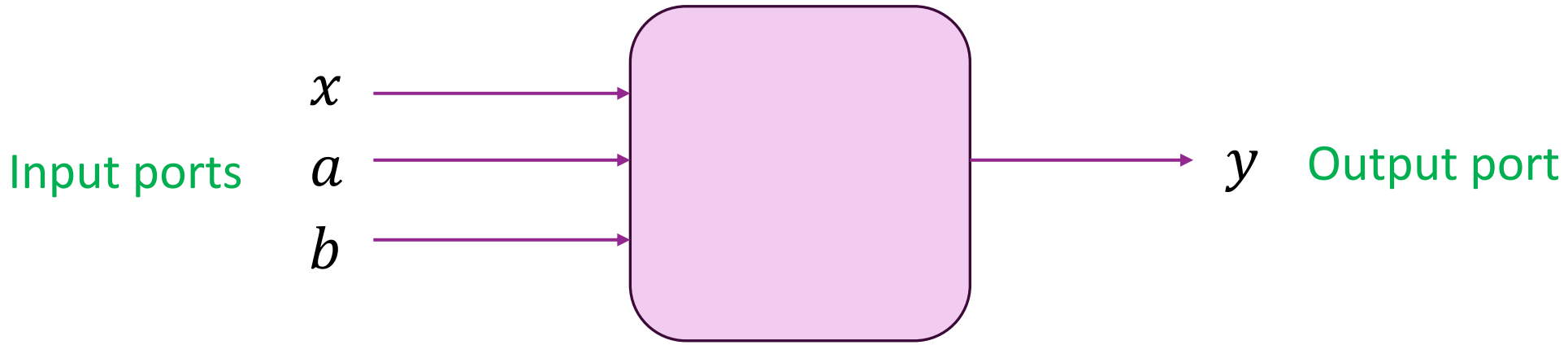
Worst-case $T_{prop} = 10$ ns

- ❑ Actual delays vary
  - Signal length, temperature, device variability
- ❑ Typically use maximum propagation delay
- ❑ Timing diagrams show max propagation delay
- ❑ X region represents:
  - Region where value cannot be guaranteed
  - Often stable before max delay
- ❑ Max propagation delay often very conservative
  - Real propagation delay may be $\ll$ maximum
  - Violating delay is usually catastrophic

Timing in a Series Circuit: ReLU + Linear

As example, consider a more complication circuit: $y = \max\{ax + b, 0\}$

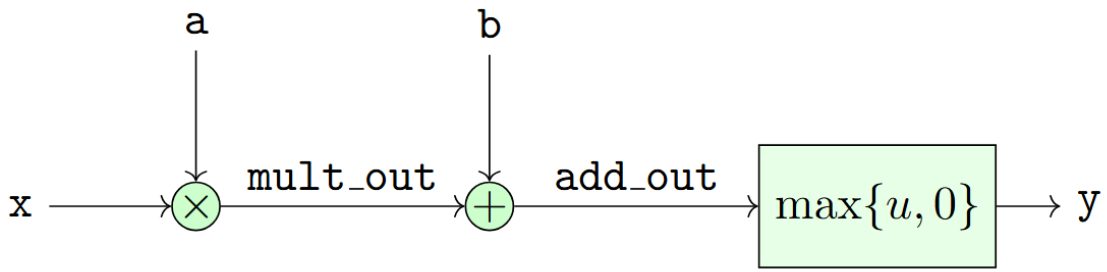
- Linear function + ReLU from Unit 2
- Used widely in machine learning



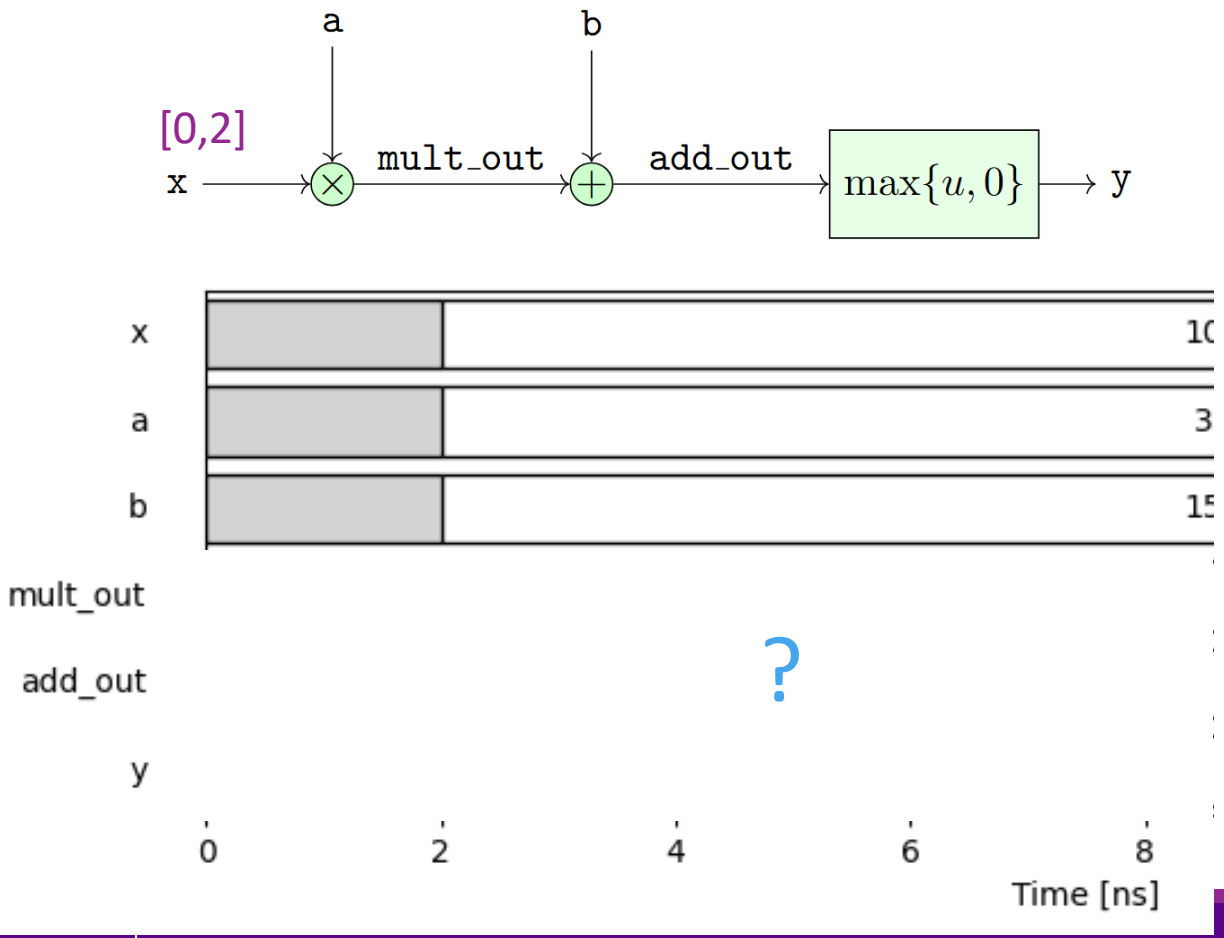
Timing Diagram for Series Connections

- Linear + ReLU can be implemented as a series connection
  - In this case, three operations one after the other
- For analysis, assume the max propagation delays in the table
- Question: What does the timing diagram look like with changes in the input?

Hardware block	Max prop delay
Multiplier	T_{mult} =3 ns
Adder	T_{add} =1.5 ns
Max operator	T_{max} =1 ns
Total	T_{tot} = 5.5 ns

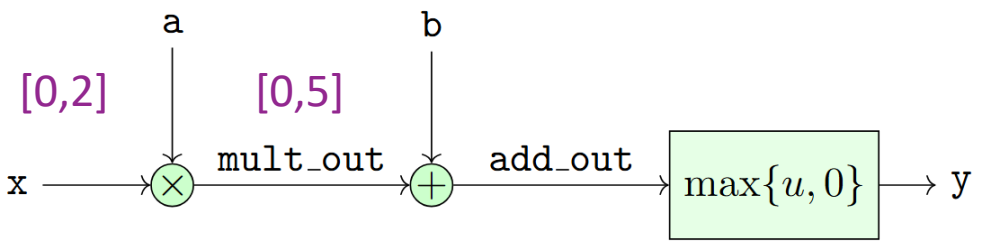


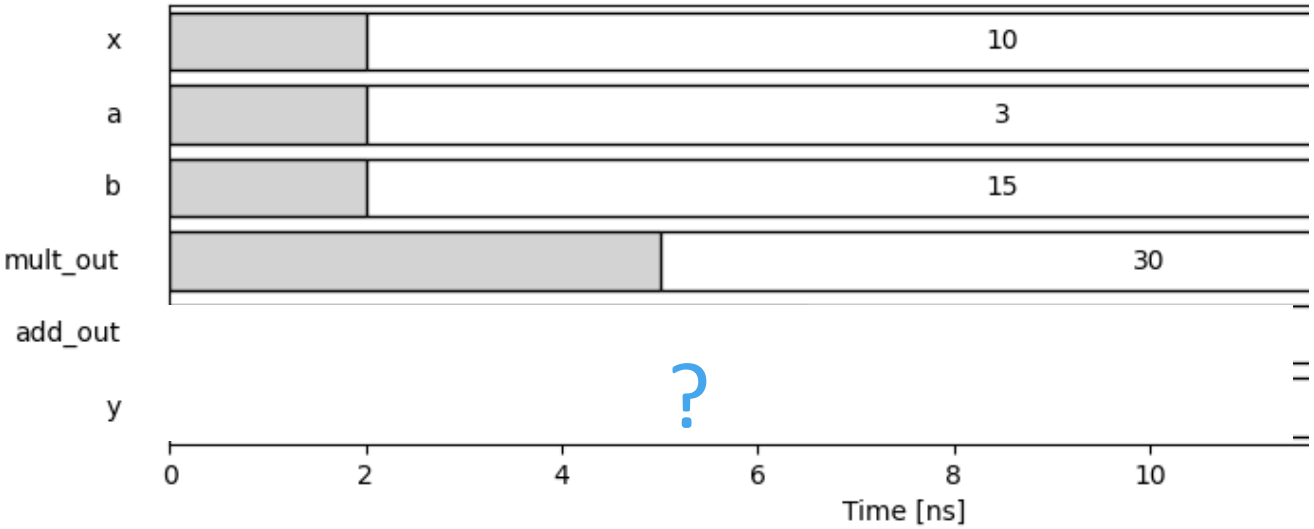
Timing Diagram of a Series



- Suppose x, a, b are invalid for $t \in [0,2]$
  - They are in a transition in this period
  - Signals are unknown X
- Mark the unknown region with violet [0,2]
- When are downstream signals stable?

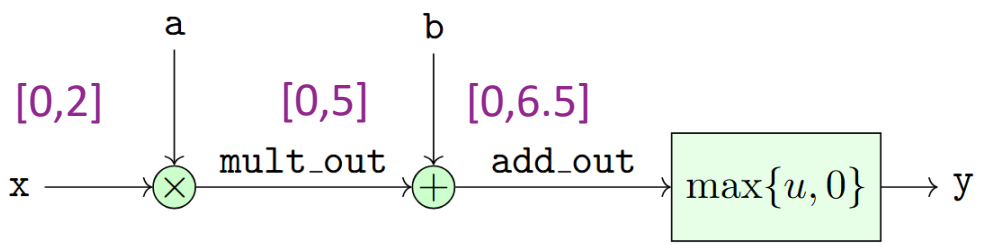
Series Timing: Initial Component

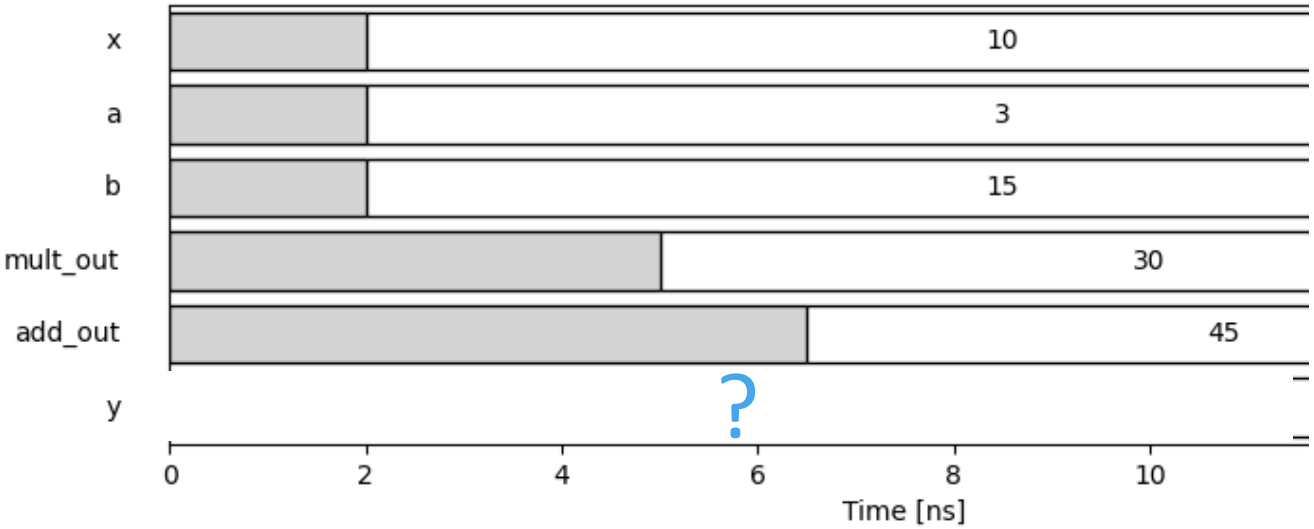




- Start with first component (multiplier)
- Inputs are invalid in [0,2]
- Max propagation delay of mult = 3 ns
- Mult_out is invalid from [0,5]
  - Invalid for 3 ns longer than input
- Final stable value:
  - Mult_out = 3*10 = 30

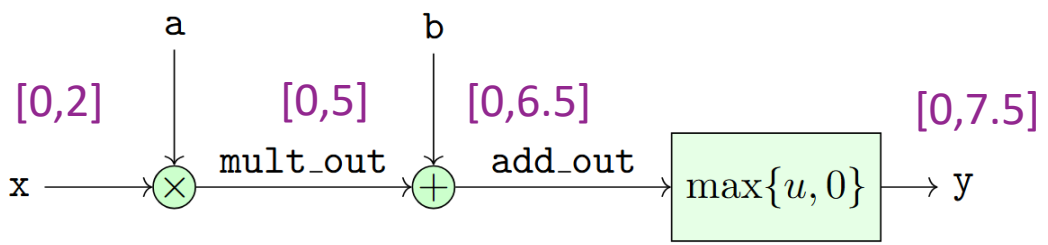
Series Timing: Second Component

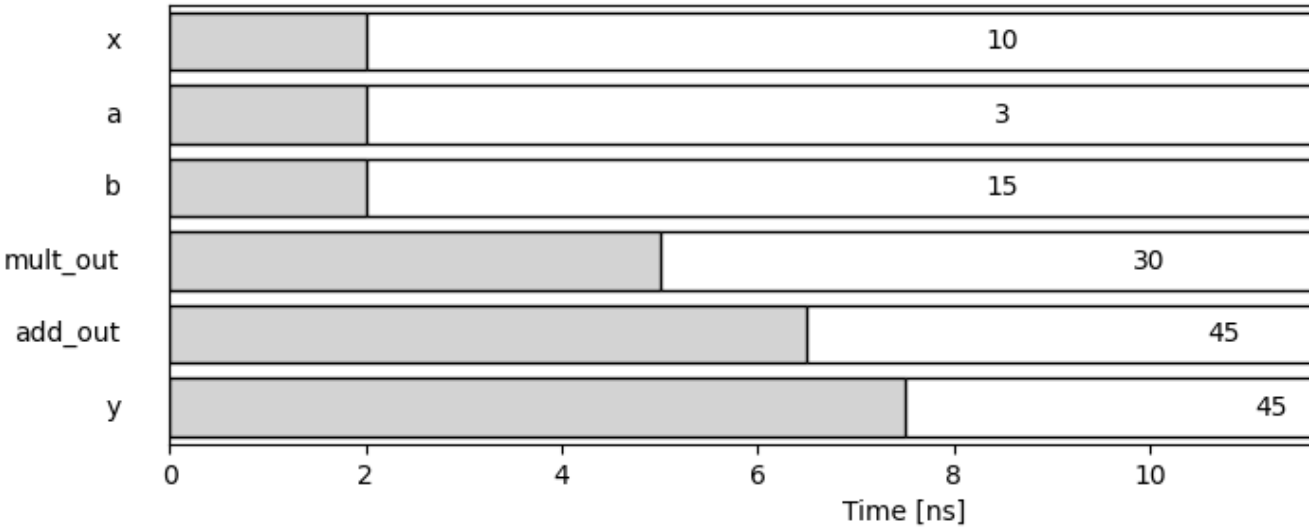




- From previous slide:
  - Mult_out is invalid from [0,5]
- Adder max propagation delay = 1.5 ns
- Hence add_out invalid in [0,6.5]
- Observe: For worst-case analysis
  - Add_out is invalid immediately when x is invalid
- Invalid propagates immediately
- Stable conditions propagate with delay

Series Timing: Third Component

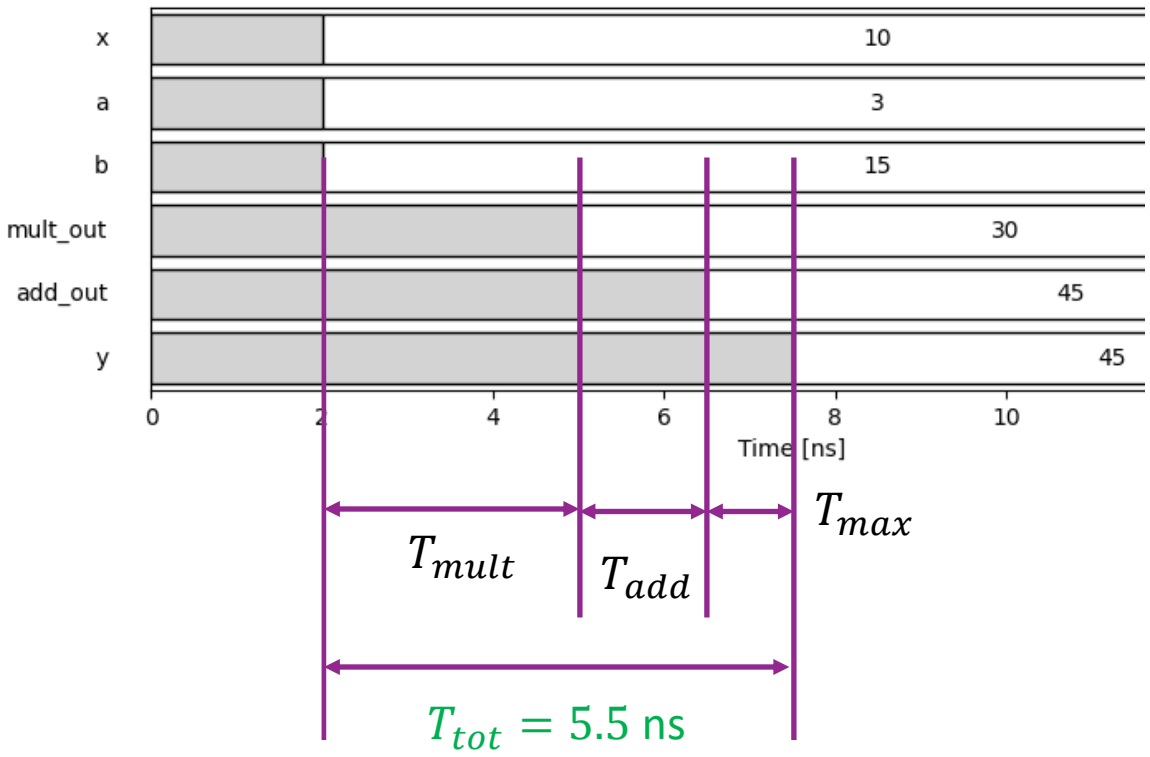




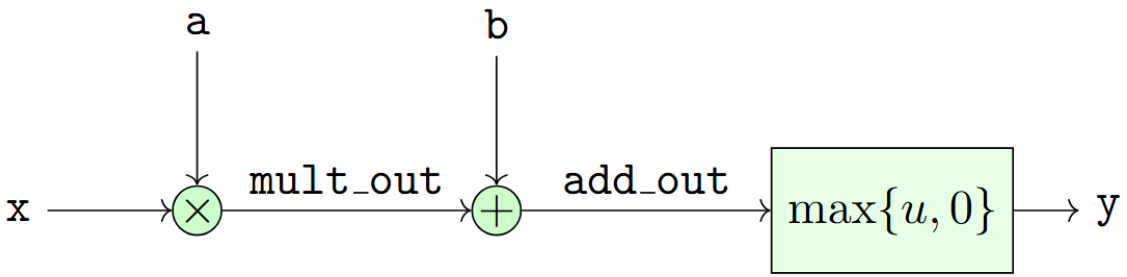
- Similar logic as add_out
- Add_out is invalid [0,6.5]
- Max logic has max prop delay = 1 ns
- Output y is invalid in [0, 7.5]

- Total propagation delay = 5.5 ns
  - Period after inputs were stable

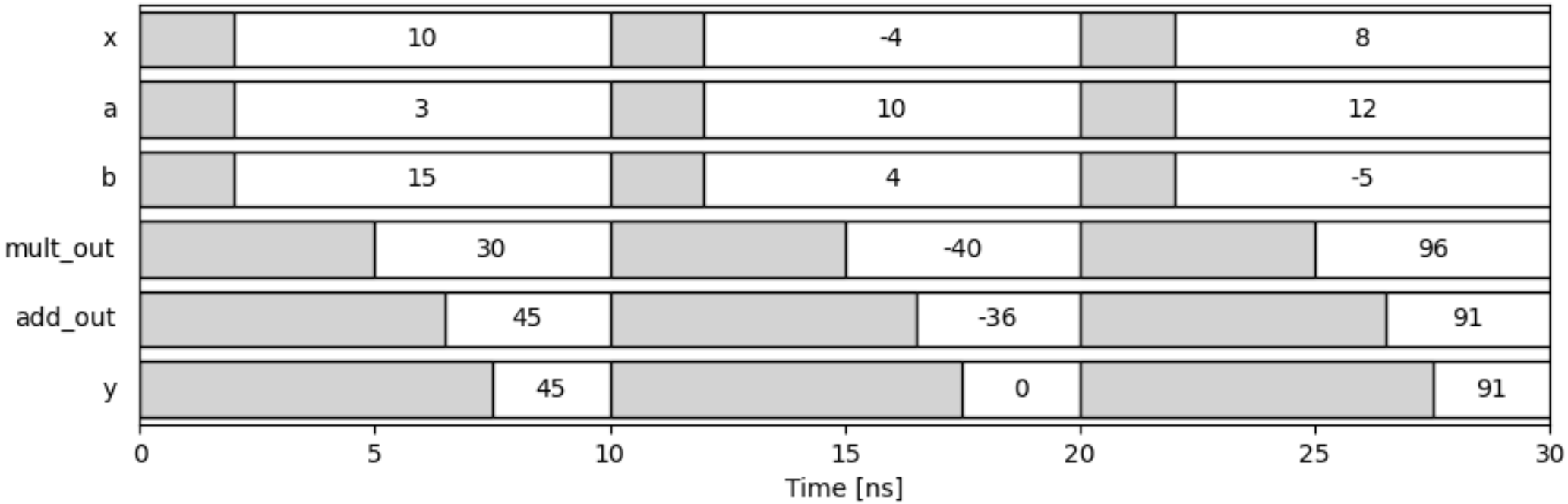
Example Total Propagation Delay



Hardware block	Example Value
Multiplier	T_{mult} =3 ns
Adder	T_{add} =1.5 ns
Max operator	T_{max} =1 ns
Total	$T_{tot} = 5.5$ ns



Example Timing Over Multiple Inputs



Example Problem

Question 2. Series timing

Consider the following SystemVerilog combinational code:

```
xsq = x*x;
y = a*xsq;
z = y + b;
```

Assume each multiplication takes 3 ns and the addition takes 2 ns. Suppose that `x` is stable at `x=0` for a long period before time 0, and then changes to `x=4` at time 10 ns, remaining stable thereafter. The variables `a=3` and `b=6` are constants. meaning that `x` changes from unknown value at 0 ns to 4 at 10 ns.

a. What are the stable values for `xsq`, `y`, and `z` before and after the change?
b. What are the time-value change points of `xsq`, `y`, and `z` starting at time = 0 ns? Make sure to indicate the periods of the unknown values (X) during the propagation delays.

Typical Propagation Delays: Low-End FPGA

Hardware block	FPGA resource	Typical range
32-bit adder	LUT + dedicated carry chain	1.5–3.0 ns
16-bit multiplier	DSP48E1 block	2.5–5.0 ns
16-bit comparator	LUT logic + carry chain	1.0–2.0 ns

- Rough delays for a low-end FPGA (e.g., Pynq Z2)
- We will explain the FPGA resources more in later units
- Routing delays not included
  - May be large for more complex designs

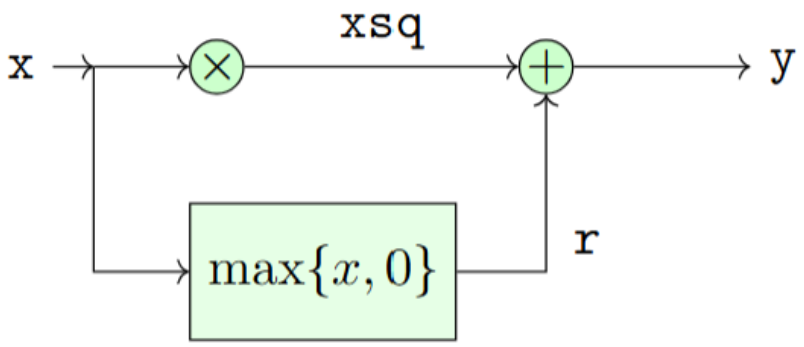
Typical Propagation Delays: ASIC

Hardware block	Typical structure	Typical range
32-bit adder	Kogge-Stone / Brent-Kung / hybrid	150-450 ps
16-bit multiplier	Wallace/Dadda tree + final adder	300-700 ps
16-bit comparator	XOR/AND/OR chain with early-out	50-300 ps

- Rough delays for TSMC 16nm
- Custom ASICs are much faster than FPGAs
  - Easily 5 to 10x faster

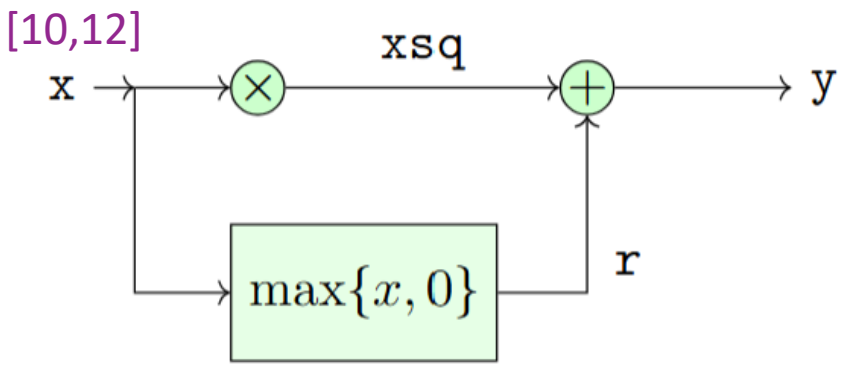
Timing Diagram with Multiple Paths

- Now consider a circuit with multiple paths
- Input to the adder has two paths:
  - One from xsq
  - Second from ReLU function r
- How does the timing diagram work?

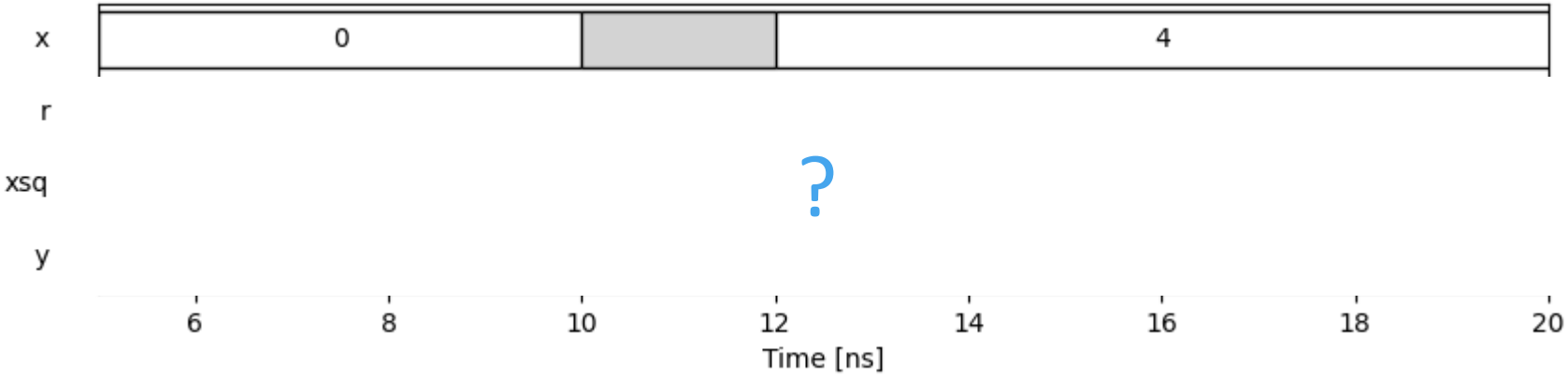



```
xsq = x * x;
r = (x > 0) ? x : 0;
y = xsq + r;
```

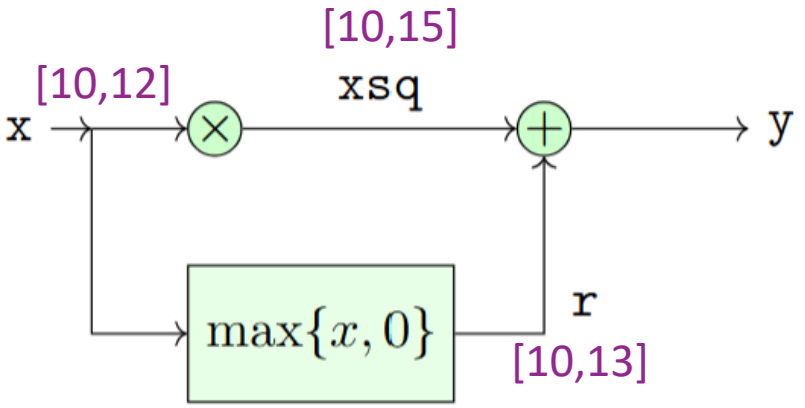
Timing Diagram: Inputs

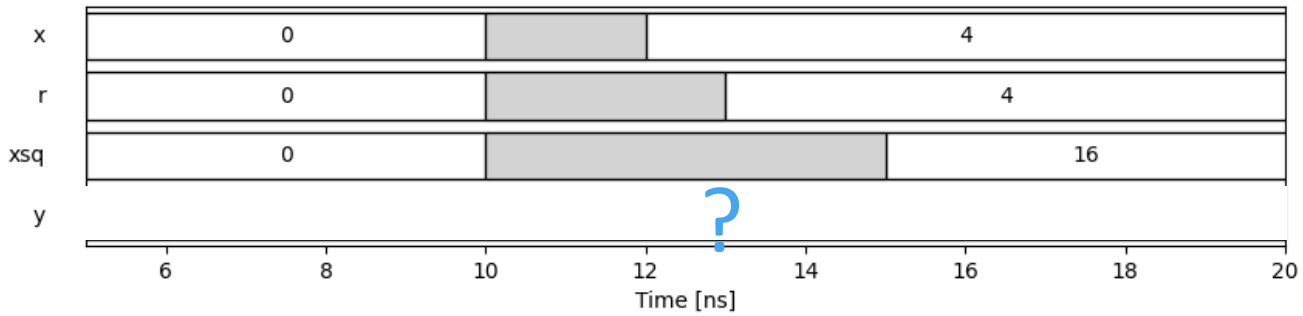


- Suppose input:
  - Transitions from $x = 0$ to 4
  - Transition interval is $t \in [10,12]$ (marked as X)



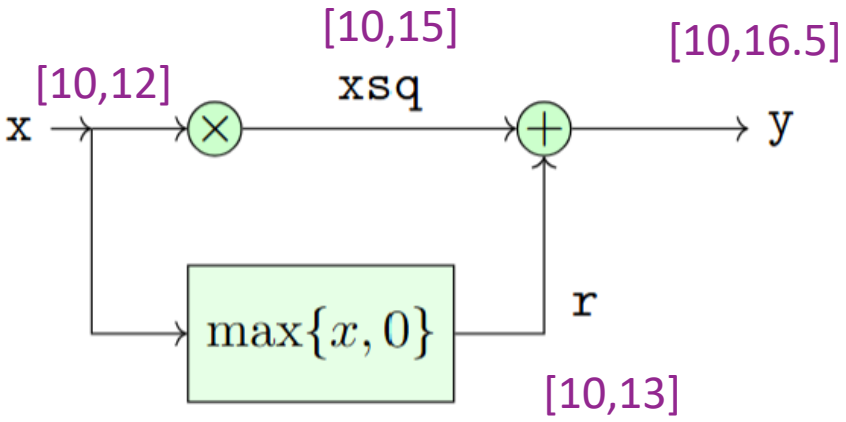
Timing Diagram: First Steps

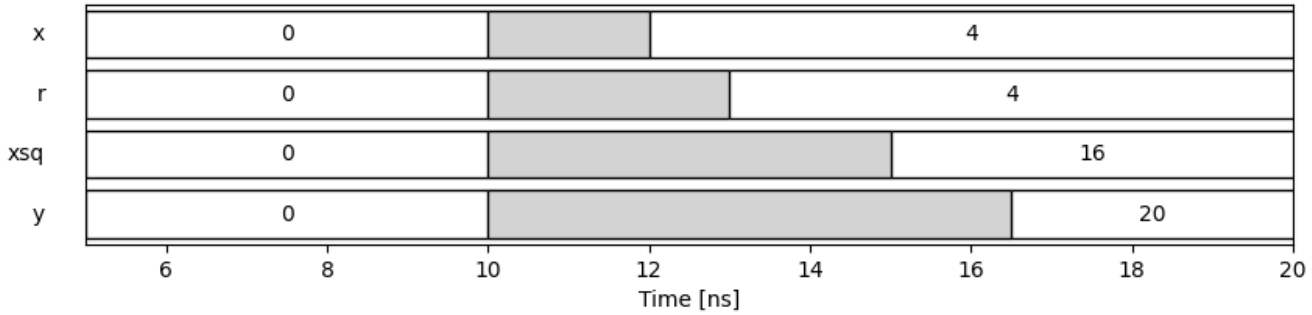




- Square output x^2
  - Multiplication max delay is 3
  - Value X from [10, 15]
  - Then comes to stable value 16
- ReLU output r
  - ReLU max delay is 1
  - Value X from [10, 13]
  - Then comes to stable value 4
- These steps operate in parallel

Multipath: Final Output





- Output y has two inputs
  - x^2 and r
- Output goes invalid when either input = X
  - Both are invalid starting at $t = 10$
  - So, y is invalid starting at $t = 10$
- Propagation delay starts when both are stable
  - Before this output cannot be guaranteed
  - x^2 is stable at $t = 15$
  - r is stable at $t = 13$
  - Propagation for adder is 1.5
  - y stable at 16.5

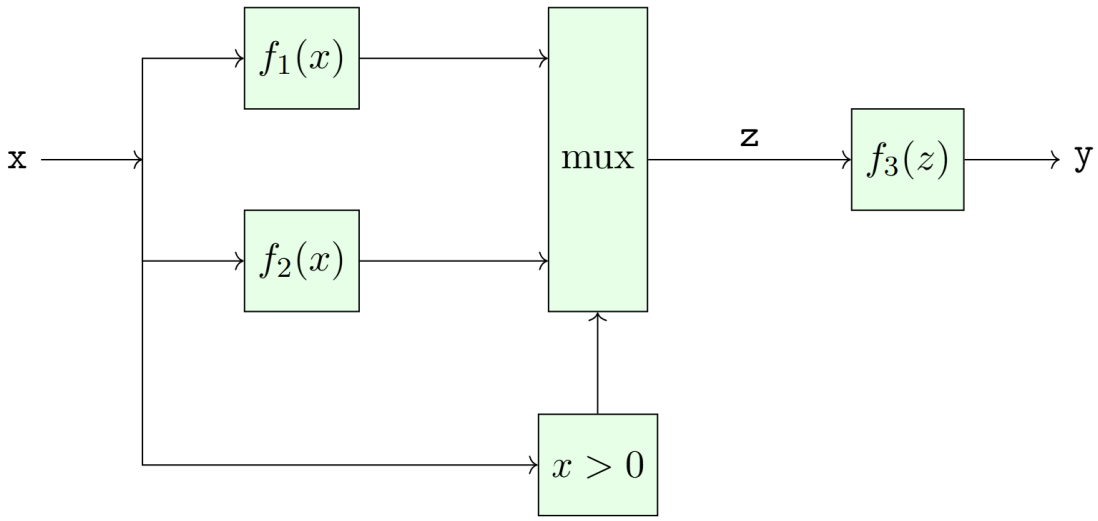
Conditional Circuits

- Consider conditional block
- How is it implemented?
- What is the max propagation delay?

```
if (x > 0) begin  // delay = 0.5
  z = func1(x);  // delay = 4
end else begin
  z = func2(x);  // delay = 2
end
y = func3(z);    // delay = 1.5
```

Implementing Conditional with Multiplexers

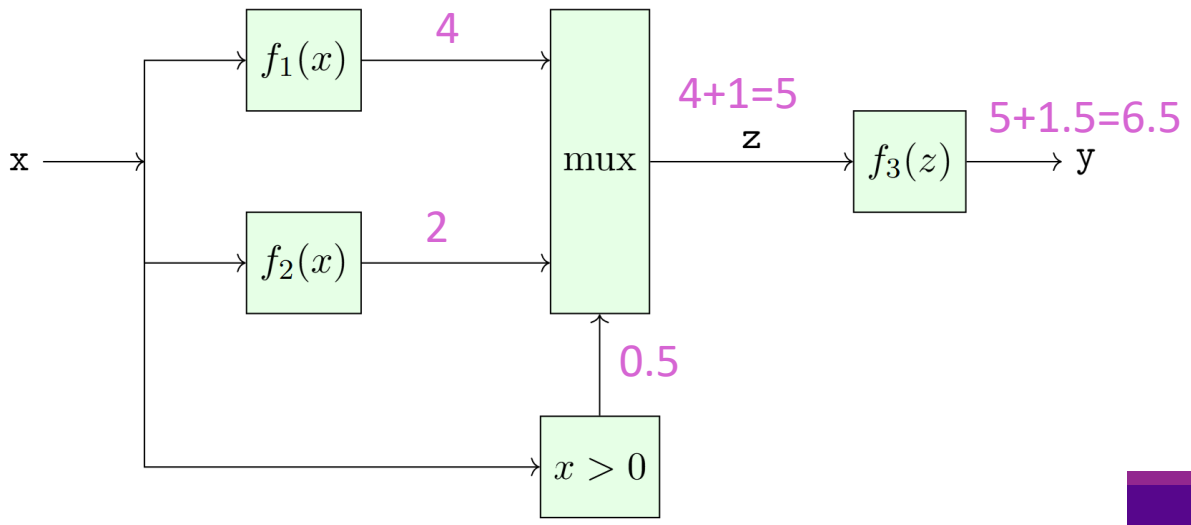
```
// mux delay = 1 ns
if (x > 0) begin  // delay = 0.5
  z = func1(x);  // delay = 4
end else begin
  z = func2(x);  // delay = 2
end
y = func3(z);    // delay = 1.5
```



- Multiplexer:
  - Circuit that selects one of a set of inputs
  - Uses a condition input
- Inputs and conditions
  - All computed in parallel
  - But each need distinct hardware
- No savings in hardware dependent on data
  - All possible inputs must be computed
- Propagation delay starts with slowest input

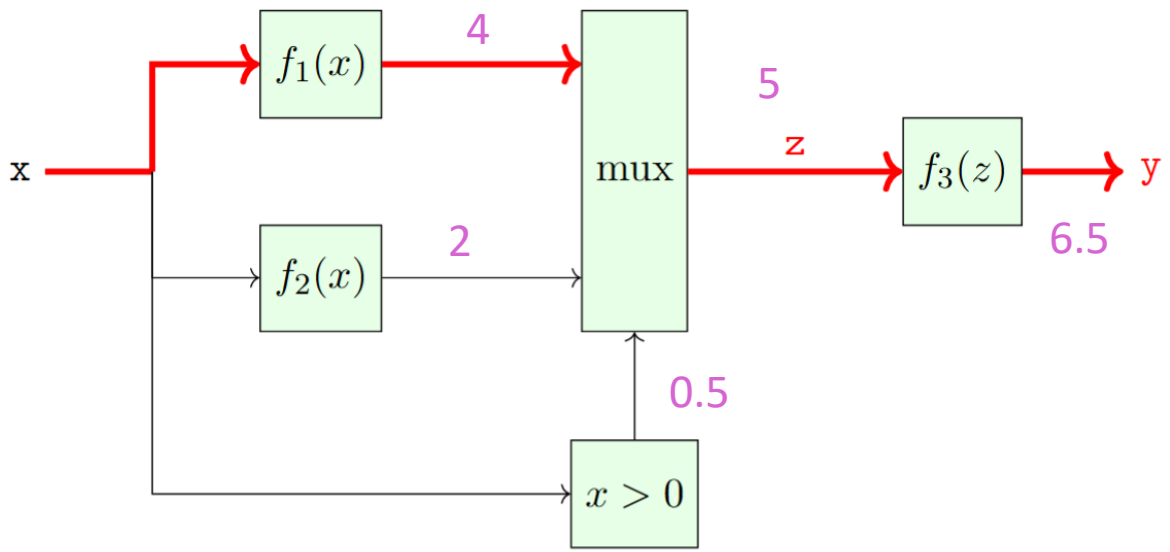
Propagation Delay with Multiplexers

```
// mux delay = 1 ns
if (x > 0) begin  // delay = 0.5
  z = func1(x);  // delay = 4
end else begin
  z = func2(x);  // delay = 2
end
y = func3(z);    // delay = 1.5
```



- Values in violet are max prop delay
- Slowest of the mux input and condition:
  - $z = f_1(x)$ branch is the slowest
  - Takes 4 ns
  - Mux adds 1 ns
  - So z is stable at 5 ns
- Output y is stable 1.5ns after z

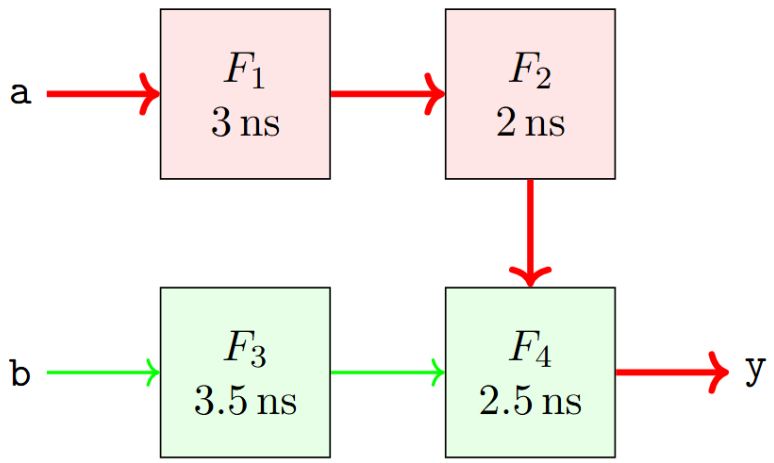
Critical Path



- Critical path:
  - The path with the largest max propagation delay
- In this case: $x \rightarrow f_1(x) \rightarrow z \rightarrow y$
- To reduce overall max propagation delay:
  - Must reduce max delay on critical path
- For example:
  - Reducing delay of $f_1(x)$ will reduce critical path
  - But reducing $f_2(x)$ does not help

Critical Path Example

- Example: For some functions $F_1, \ldots, F_4$
  - $x_1 = F_2(F_1(a))$
  - $x_2 = F_3(b)$
  - $y = F_4(x_1, x_2)$
- Path 1 ($F_1 \rightarrow F_2 \rightarrow F_4$):
  - Delay = 3+2+2.5 = 7.5 ns
  - Longest = Critical path
- Path 2 ($F_3 + F_4$):
  - Delay = 3.5 + 2.5 = 6 ns
  - Shorter = Non-critical

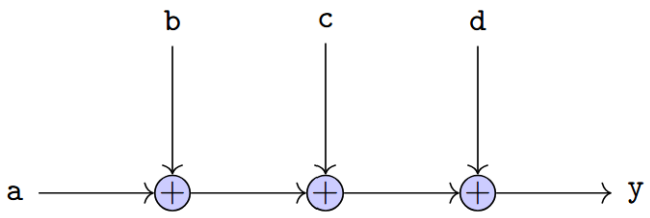


Delay Depends on Implementation

- Example: Consider $y = a + b + c + d$
- Suppose each add takes D seconds
- Synthesis tools try to find low delay implementations

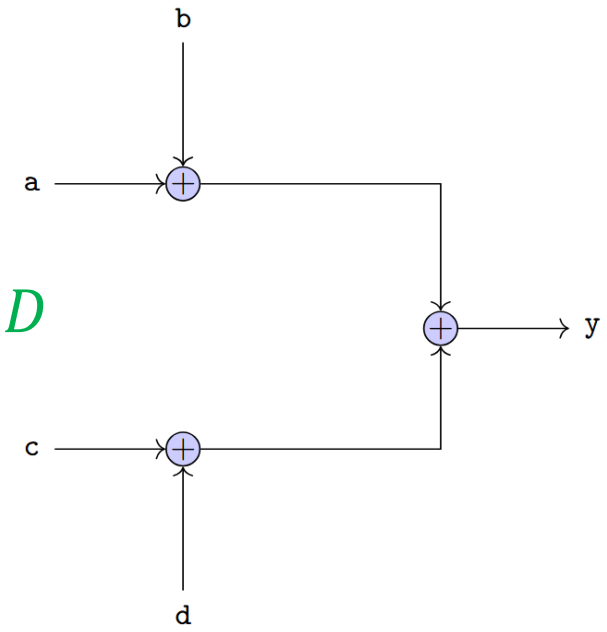
Serial implementation

Prop delay = $3D$



Parallel

Prop delay =$2D$

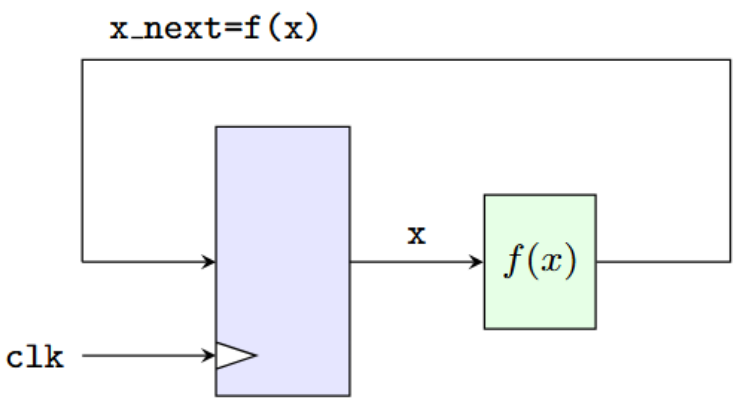


Outline

- ❑ Static Timing Analysis in Combinational Logic
- ❑ Static Timing Analysis in Sequential Logic
- ❑ Sequential Logic Timing Diagrams
- ❑ VCD Files and Timing Diagrams from Vitis

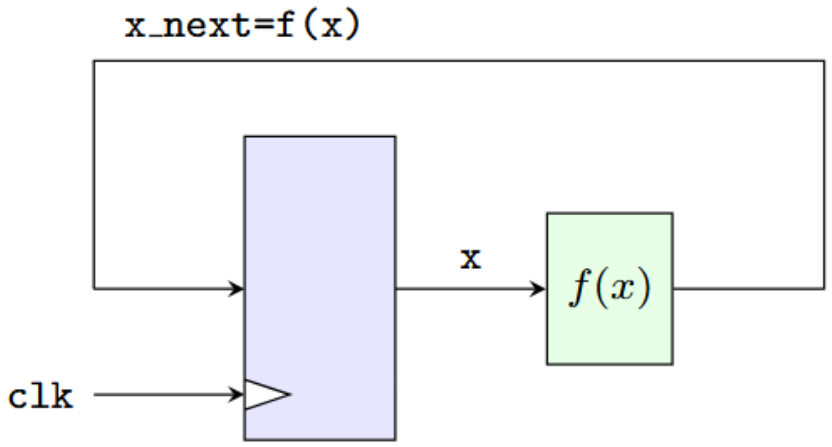
Synchronous Logic

- We now perform static timing analysis for synchronous logic
- This unit, we assume
  - All registers clocked on a common positive clock edge
  - All registers are perfectly synchronized (no clock skew)
  - No asynchronous reset




```
always_comb begin
  x_next = f(x);
end
always_ff @(posedge clk) begin
  x <= x_next;
end
```

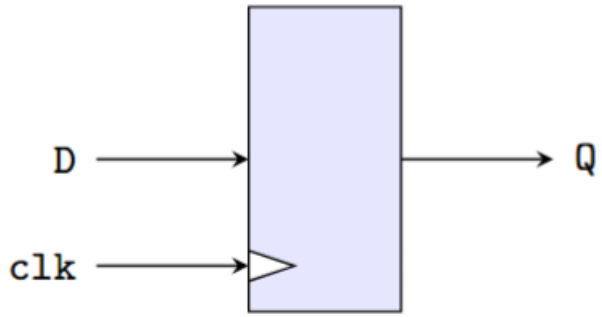
STA for Synchronous Logic Overview

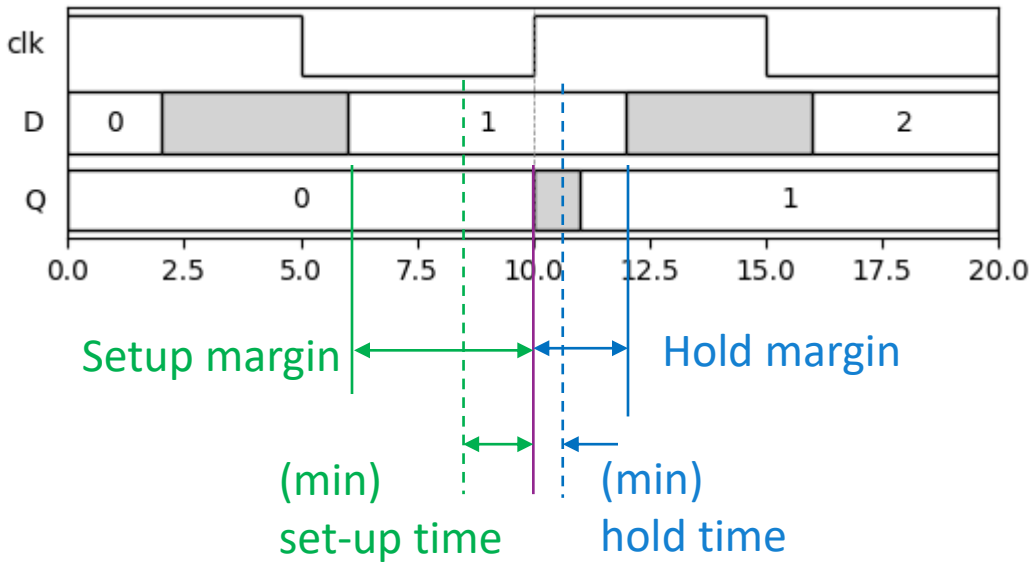


- STA considers all register-to-register paths
  - Every register output to register input
- Tests two conditions
- No paths arrive too late
  - Signals must arrive before next transition
  - Must meet a set-up time and clock-to-Q time
- No paths arrive too early
  - Min delay > hold time
  - Registers inputs must be stable for a hold period

Set-Up and Hold Time

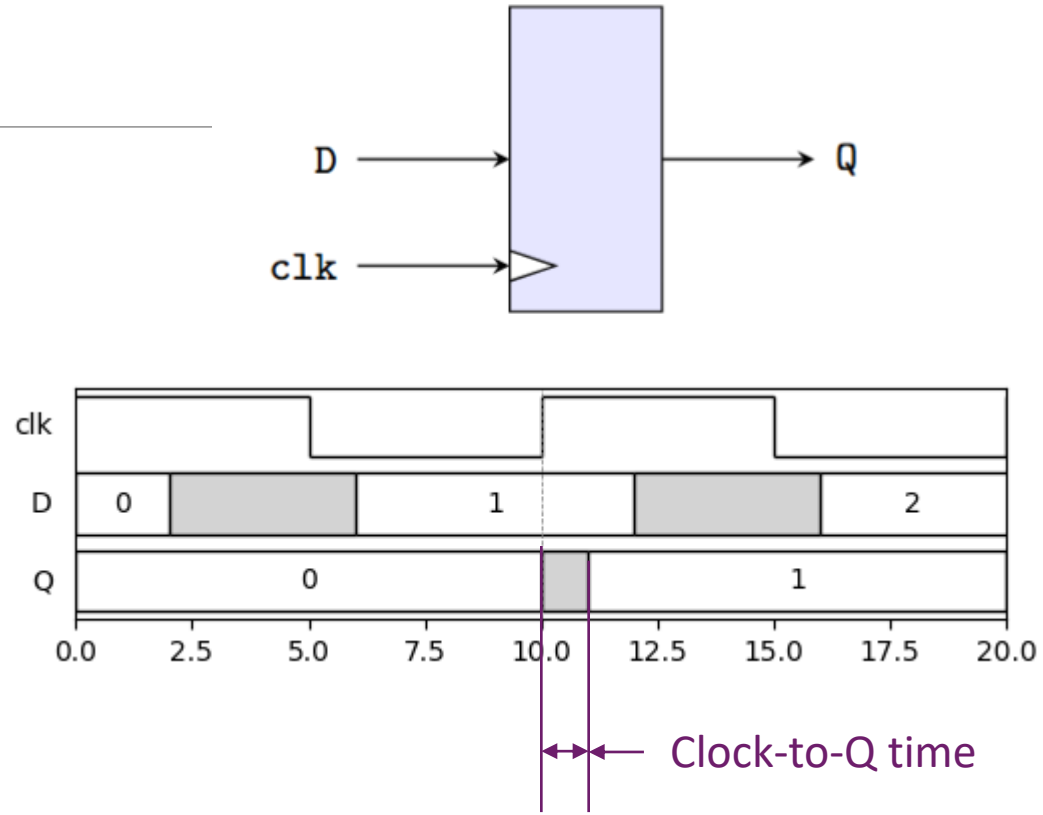
- Registers typically implemented with a flip-flop
  - Sometimes called D-Q flip-flop
- Stable operation requires:
  - Inputs are stable before and after clock edge
- Set up margin:
  - Time from D stable to rising clock edge
  - Requirement: Set-up margin > set-up time
- Hold margin:
  - Time D stable after rising clock edge
  - Requirement: Hold margin > hold time





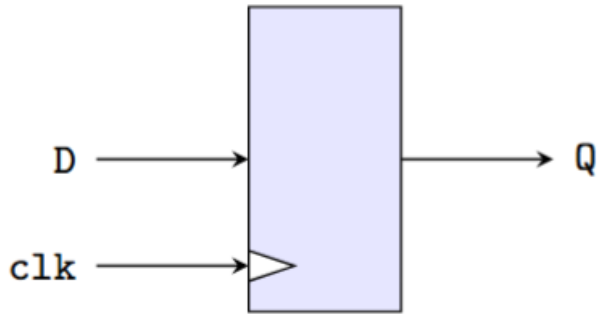
Clock-to-Q time

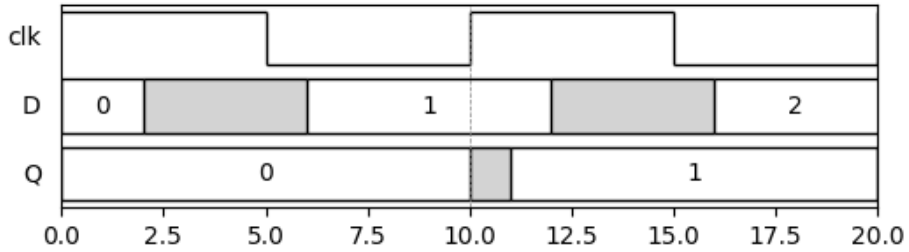
- Clock-to-Q time
  - Time from rising edge to stable output Q
- Sometimes loosely called register delay
- Physically time to transfer input to output



Typical Times

Technology	Clock-to-Q	Setup Time	Hold Time	Typical Clock Period
Low-end FPGA (e.g., Pynq / Zynq-7000 / Artix-7)	**300–600 ps**	**150–300 ps**	**0–150 ps**	**5–10 ns** (100–200 MHz)
High-end FPGA (e.g., Xilinx UltraScale+, Intel Stratix 10)**	**150–300 ps**	**100–200 ps**	**0–120 ps**	**1.5–3 ns** (300–650 MHz)
ASIC (modern nodes, e.g., 7–28 nm)	**20–80 ps**	**20–60 ps**	**–20 to +40 ps**	**0.3–1.0 ns** (1–3+ GHz)

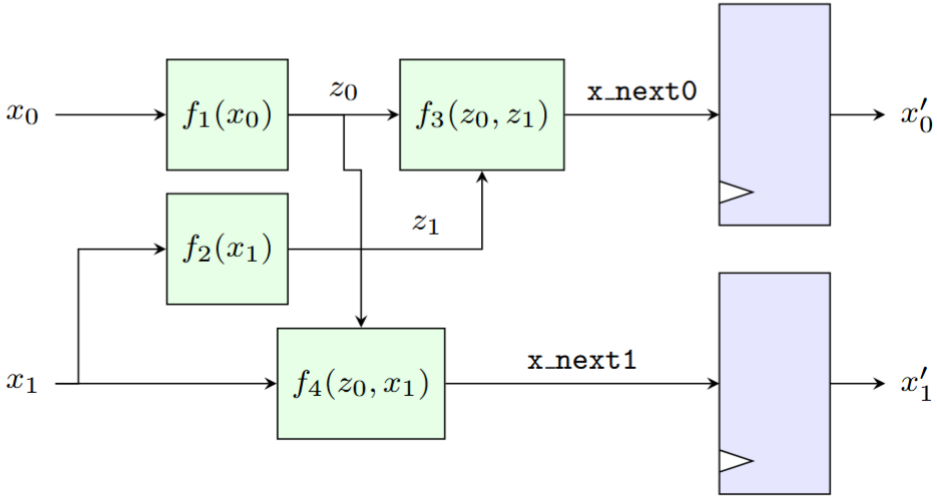




Note: Modern nodes may have a negative hold time!

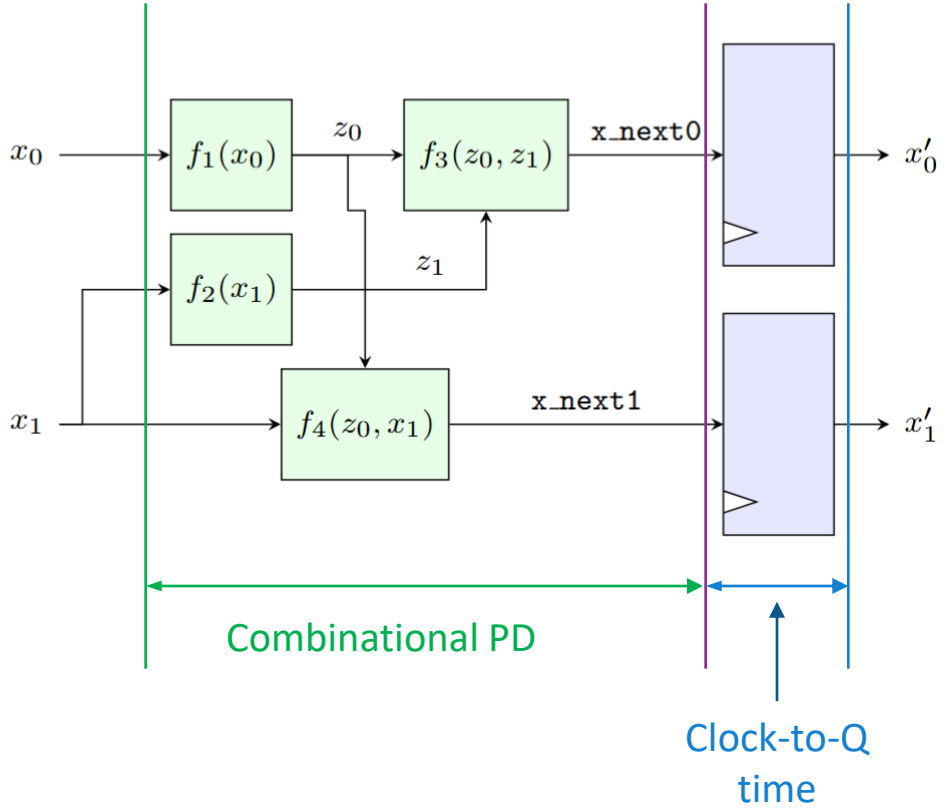
Register-to-Register Paths

```
always_comb begin
  z0 = func1(x0);
  z1 = func2(x1);
  x_next0 = func3(z0, z1);
  x_next1 = func4(z0, x1);
end
always_ff @(posedge clk) begin
  x0 <= x_next0;
  x1 <= x_next1;
end
```



- Register-to-register paths
  - Start at any register output
  - End at any register output
- Can be visualized on a dataflow graph
  - Maps logics and register for each update
  - Primed values x'_0, x'_1 represent values in *next* cycle
- In this example, there are four paths
  - $x_0 \rightarrow f_1 \rightarrow f_3 \rightarrow x'_0$
  - $x_0 \rightarrow f_1 \rightarrow f_4 \rightarrow x'_1$
  - $x_1 \rightarrow f_2 \rightarrow f_3 \rightarrow x'_0$
  - $x_1 \rightarrow f_4 \rightarrow x'_1$

STA Analysis: Verifying the Set-Up Time



- For now: Assume min hold time condition is met
  - We will address this later
- Must verify the set-up time condition:
  - Max path delay $\leq T_{clk} - T_{setup}$
  - All paths arrive within set up period before clock edge
- Path delay has two components
  - T_{PD}= Combinational propagation delay
  - T_{clk-Q} = One clock-to-Q delay
- Now $\max T_{PD} = T_{crit}$ = Max PD of critical path
  - Measured in combinational circuit
- Hence, to meet timing we need:

$$T_{crit} + T_{clk-Q} + T_{setup} \leq T_{clk}$$

Meeting or Failing Timing

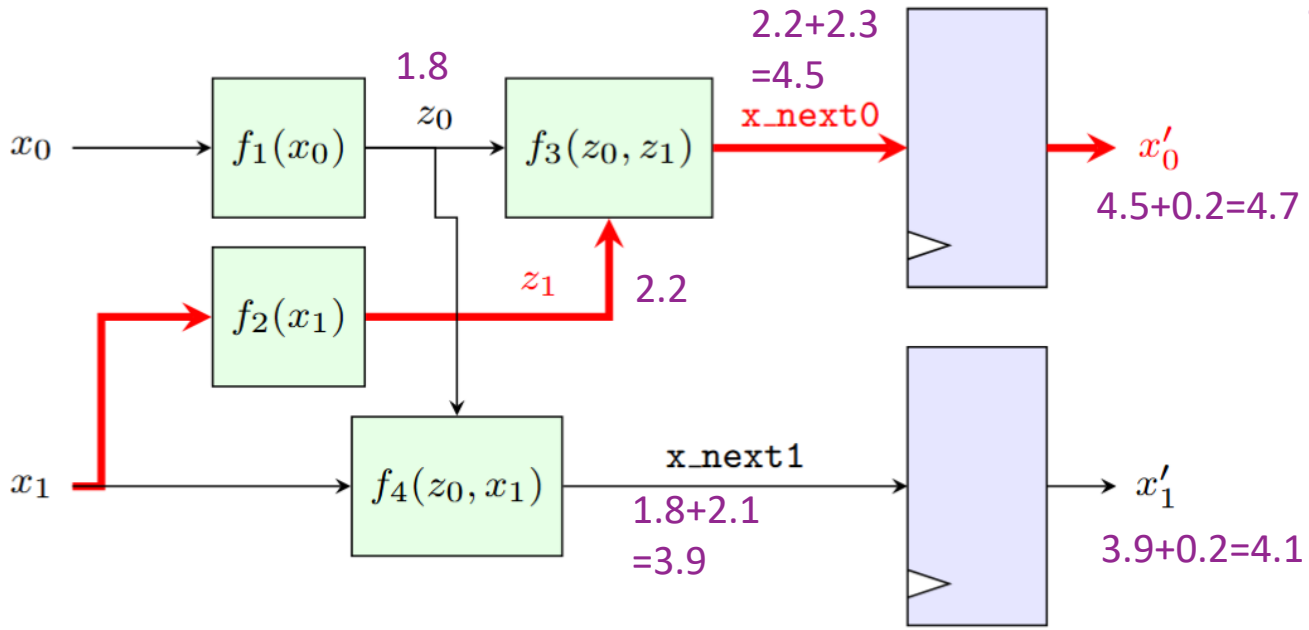
- From previous slide, assuming minimum hold time condition is met:

To meet timing: $T_{clk} > T_{crit} + T_{setup} + T_{CLK-Q}$

  - T_{clk} = clock period, $f_{clk} = \frac{1}{T_{clk}}$ = clock frequency
  - T_{crit} = delay of critical path in combinational logic
  - T_{setup} = min set up time, T_{CLK-Q} = Clock-to-Q time

- If condition is satisfied, say system meets timing
- Otherwise, system fails timing / timing violation

Example: Calculating Critical Path

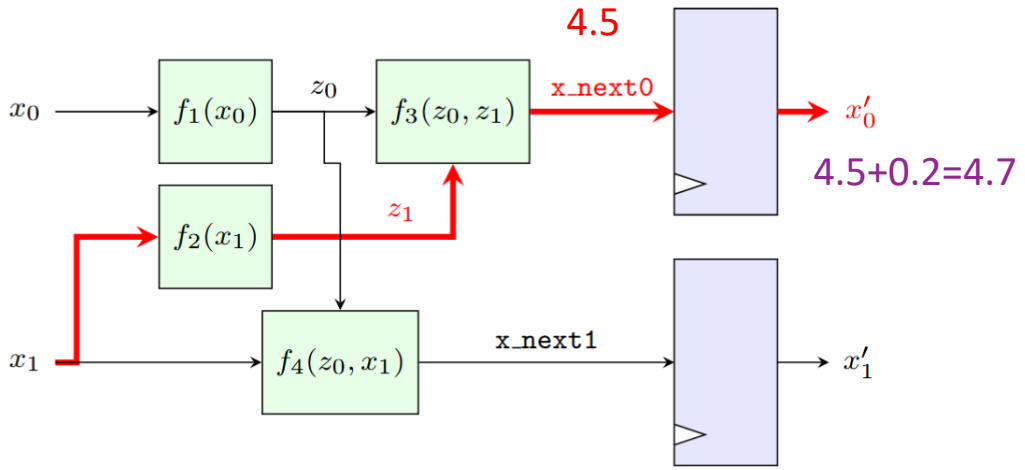


- Max propagation delay
  - Can be computed recursively
  - Identical to combinational section

Component	Value [ns]
f_1 PD	1.8
f_2 PD	2.2
f_3 PD	2.1
f_4 PD	2.3
T_{clk-Q}	0.2
T_{setup}	0.1

Example: Maximum Clock Frequency

Component	Value [ns]
T_{crit}	4.5
T_{clk-Q}	0.2
T_{setup}	0.1
T_{clk} min	4.8



- From previous slide:
  - Critical path delay in combination logic: $T_{crit} = 4.5$ ns
- Therefore, to meet timing:

$$T_{clk} \geq T_{crit} + T_{CLK-Q} + T_{setup}$$
$$= 4.5 + 0.2 + 0.1 = 4.8 \text{ ns}$$

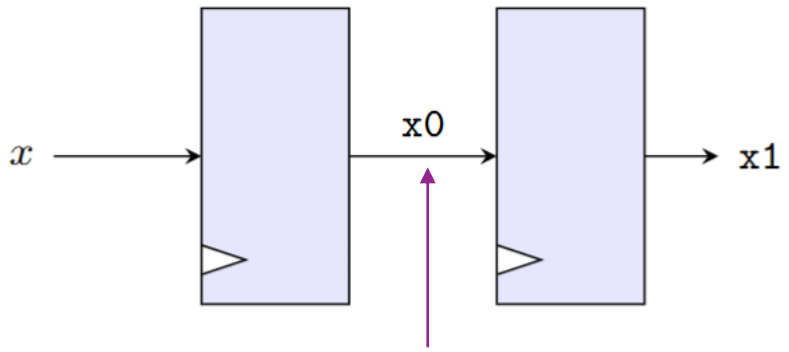
- Max clock frequency

$$f_{clk} = \frac{1}{T_{clk}} \leq 208 \text{ MHz}$$

Meeting the Min Hold Time Constraint

- Consider circuit to the right
  - A simple delay line
- Naïve implementation
  - Very short path: Output $x_0 \rightarrow$ input x_1
  - Could violate minimum hold time
- Typical solved in place & route
  - Physical design will insert delay elements
- With reasonable synchronous design:
  - RTL design does not need to worry about this constraint

```
always_ff @(posedge clk) begin
  x0 <= x;
  x1 <= x0;
end
```

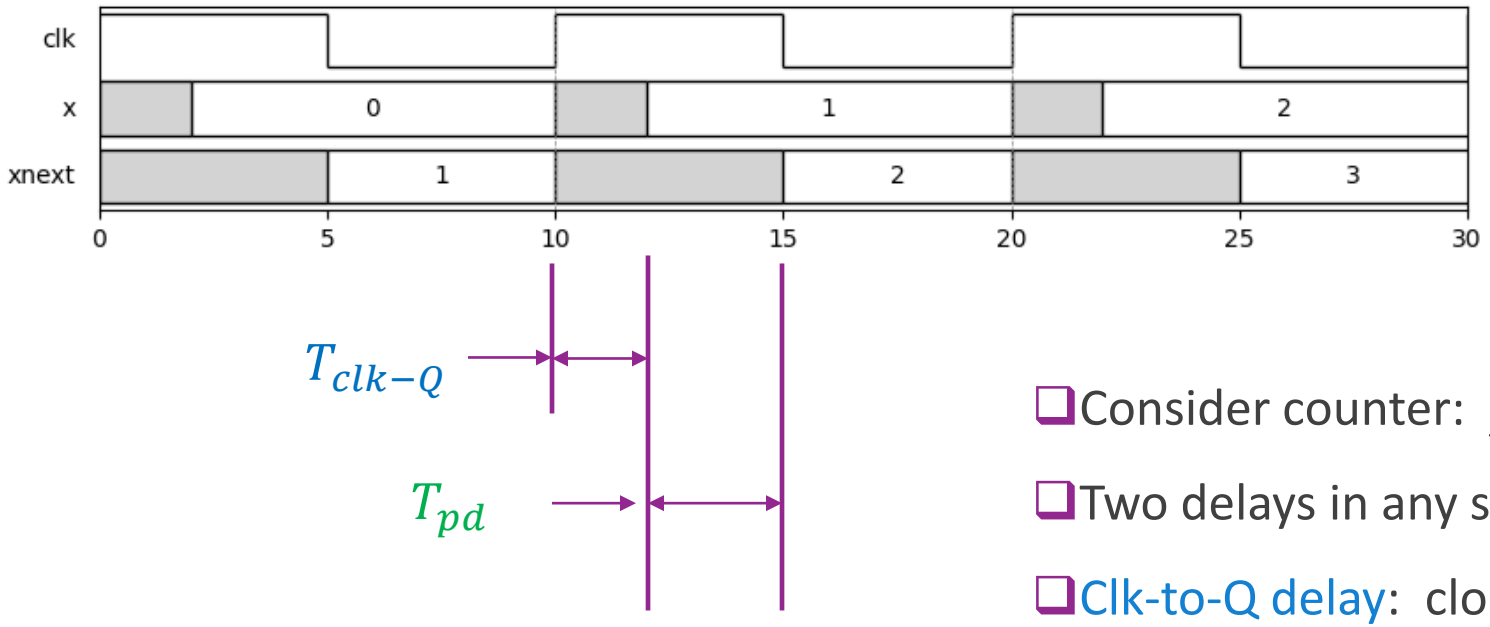


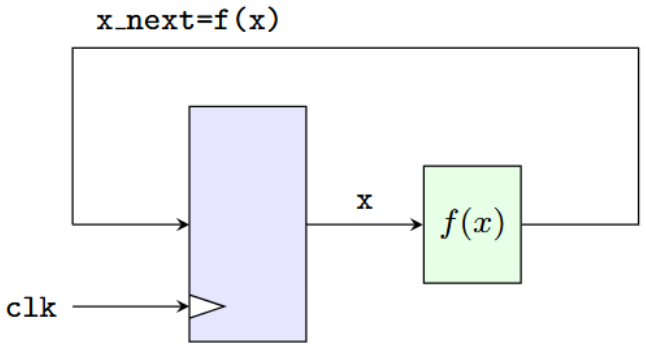
Short path.
Could violate min hold

Outline

- ❑ Static Timing Analysis in Combinational Logic
- ❑ Static Timing Analysis in Sequential Logic
- ❑ Sequential Logic Timing Diagrams
- ❑ VCD Files and Timing Diagrams from Vitis

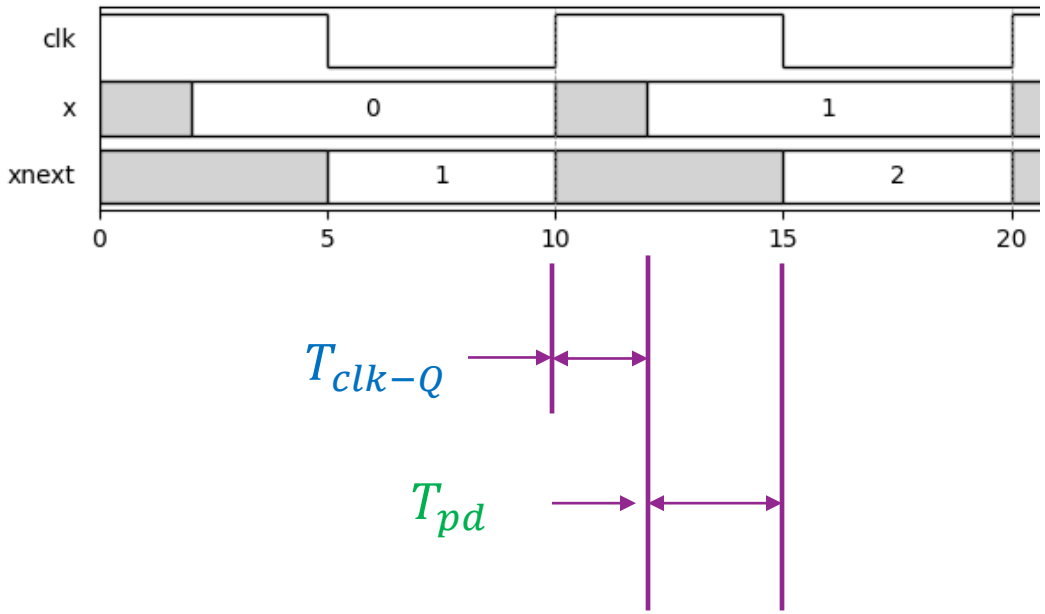
Synchronous Logic Detailed Timing Diagram





- Consider counter: $f(x) = x + 1$
- Two delays in any synchronous circuit
- Clk-to-Q delay: clock rising edge → register output
  - This example shown as 2 ns
- Combinational PD: register output → next value
  - This example shown as 3 ns

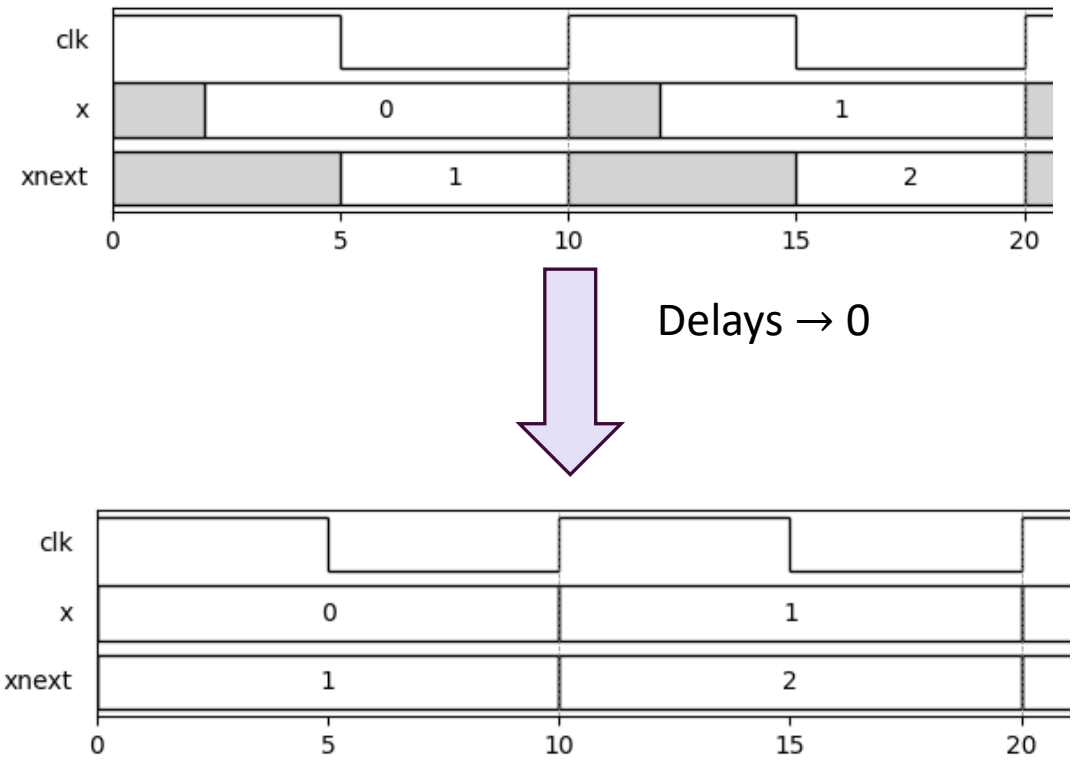
Detailed Timing Diagram



- Timing diagram drawn called detailed diagram
  - Sometimes called the real timing diagram
- Shows all details:
  - Register delays
  - All propagation delays in combinational paths
- Good for reasoning about precise timing
  - Can compute computation speed, throughput, …
- But too many details to understand functionality

Abstract or Cycle Accurate Timing Diagram

Detailed timing diagram



Abstract timing diagram

- Abstract timing diagram
  - Also called cycle-accurate diagram
  - All delays → 0
  - Ignores computation time
- No unknown (X or ?) regions
- Simpler to see functionality
- Values in each cycle represent
  - Value immediately before rising edge at end of cycle

ReLU + Linear Synchronous Module

- Now consider synchronous version of ReLU + linear
- Identical to Unit 2
- We add two inputs to ports
  - Clock signal
  - Reset on negative edge

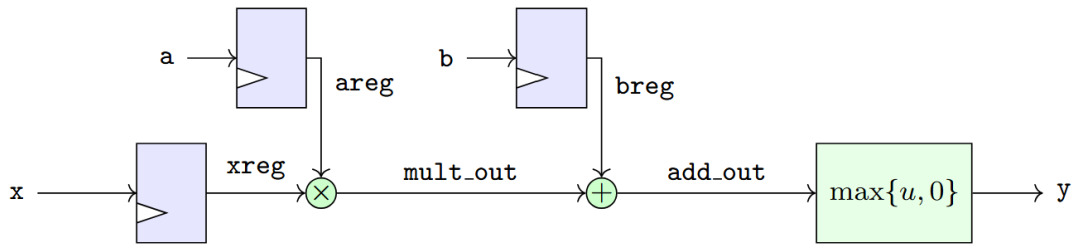
```
module relu_lin (
  input logic clk,
  input logic rst_n,
  input logic int x,
  input logic int a,
  input logic int b,
  output logic int y
);
```

Clock

Reset

Synchronous Implementation

- Use identical implementation as in Unit 2
- Next values computed in always_comb block

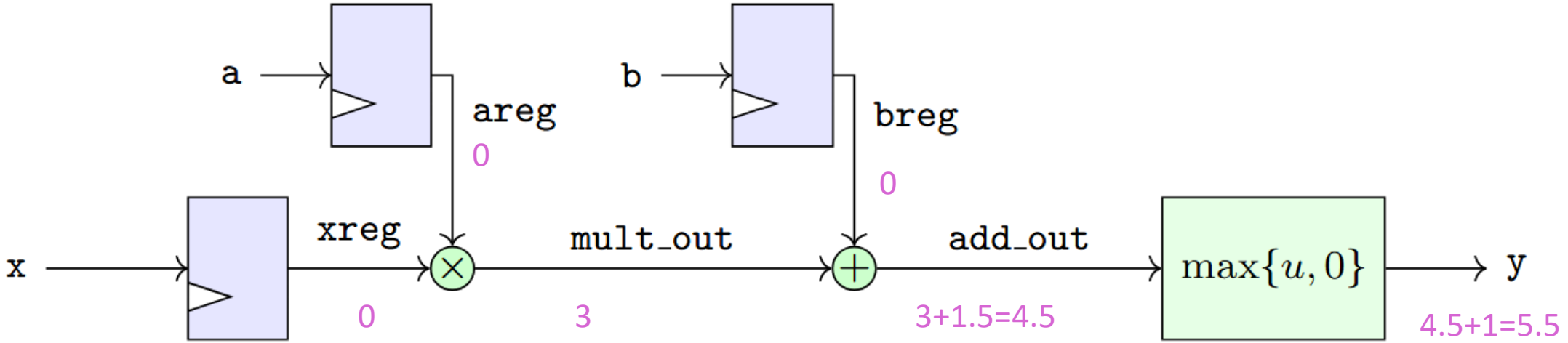



```
  int x_reg, a_reg, b_reg;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      x_reg <= 0;
      a_reg <= 0;
      b_reg <= 0;
    end else begin
      x_reg <= x;
      a_reg <= a;
      b_reg <= b;
    end
  end
  always_comb begin
    logic int mult_out, add_out;
    mult_out = x_reg * a_reg;
    add_out = mult_out + b_reg;
    y = (add_out > 0) ? add_out : 0;
  end
endmodule
```

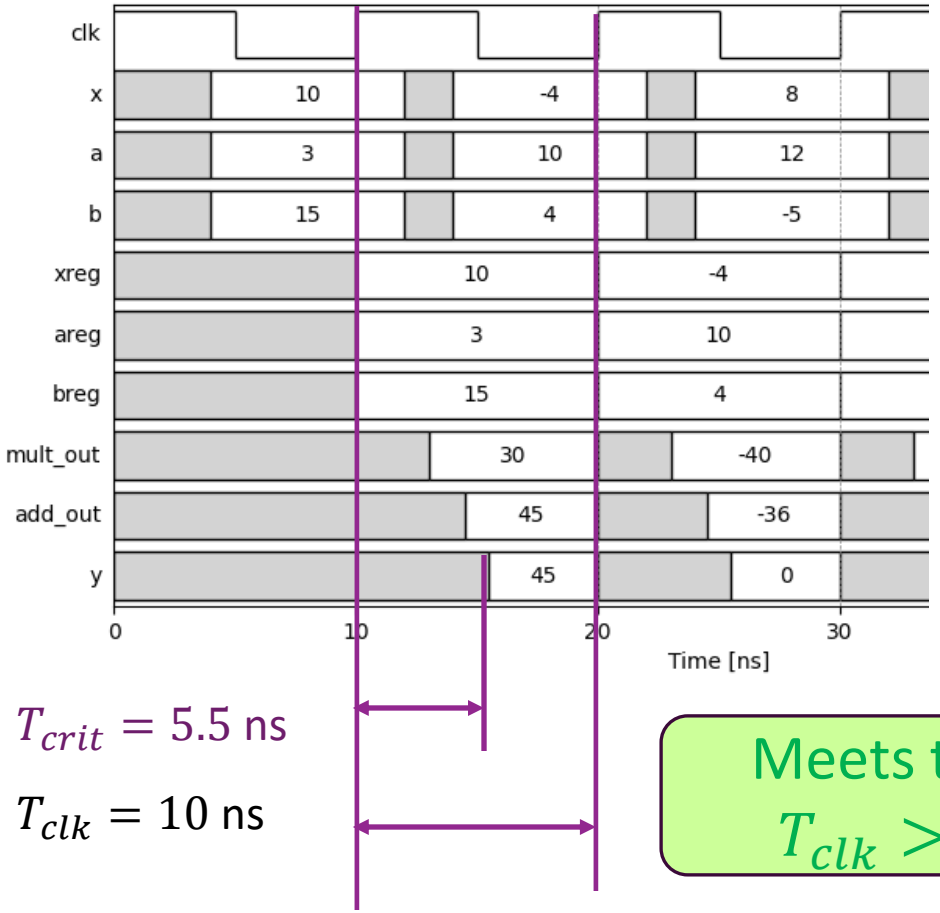
STA

- Assume the propagation delays in table
- Delay in each stage shown in violet
- Ignore CLK-Q time

Hardware block	Max prop delay
Register	0 ns
Multiplier	T_{mult} =3 ns
Adder	T_{add} =1.5 ns
Max operator	T_{max} =1 ns
Total =critical path	T_{crit} = 5.5 ns



ReLU+Linear Timing Diagram

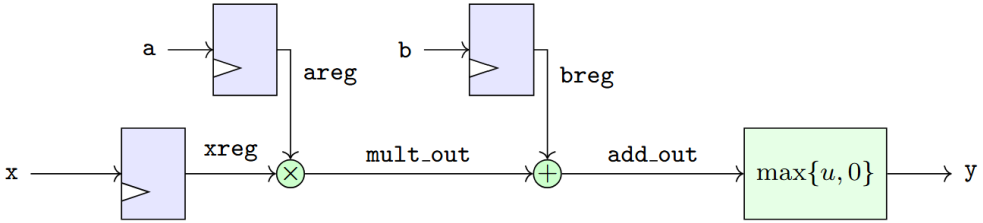


$T_{crit} = 5.5$ ns

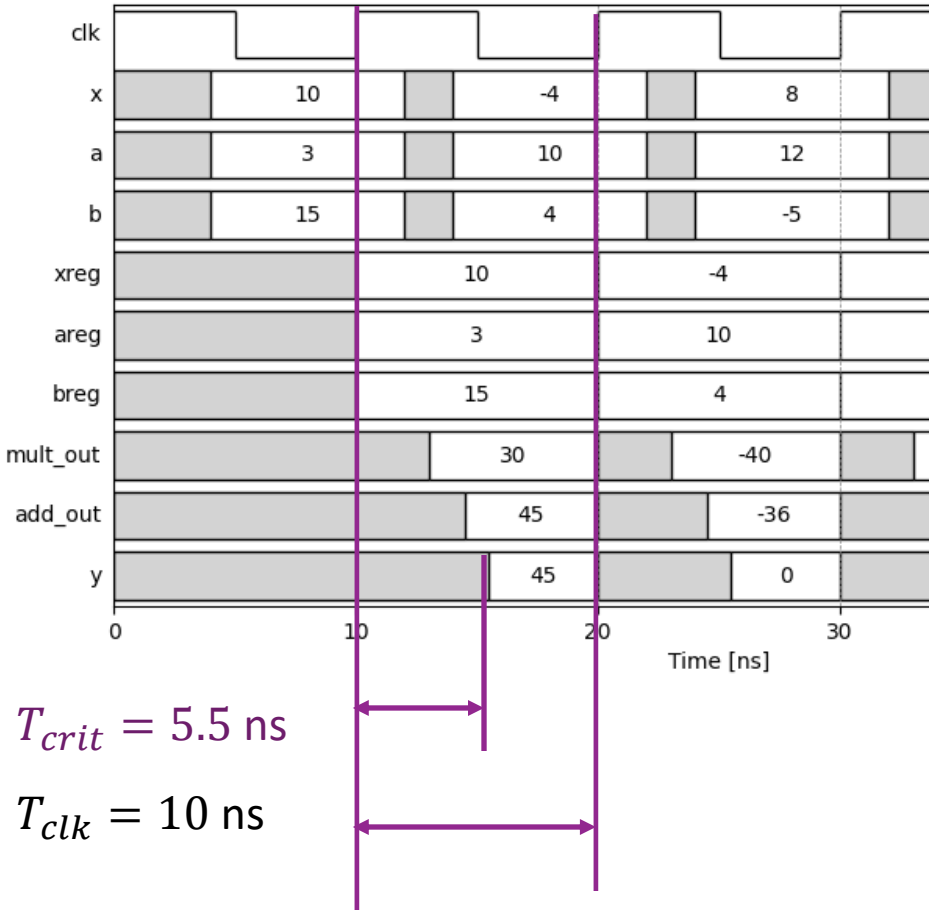
$T_{clk} = 10$ ns

Meets timing:
$T_{clk} > T_{crit}$

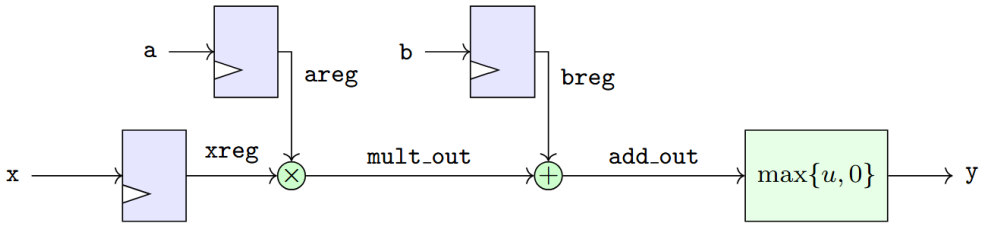
Hardware block	Example Value
Multiplier	T_{mult} =3 ns
Adder	T_{add} =1.5 ns
Max operator	T_{max} =1 ns
Total =critical path	$T_{crit} = 5.5$ ns



Max Clock Frequency Example

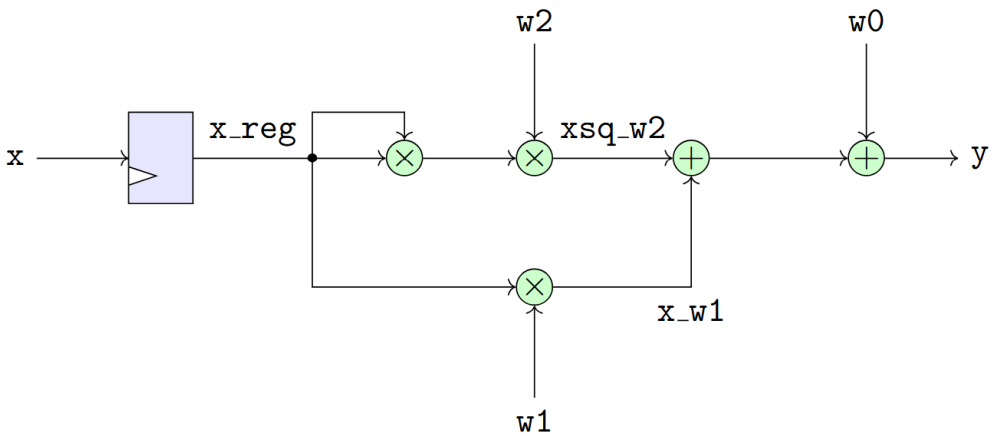


- Max clock frequency in our example:
  - Min $T_{clk} = T_{crit} = 5.5$ ns
  - Max clock frequency = $\frac{1}{5.5} \cong 180$ MHz



Quadratic Example

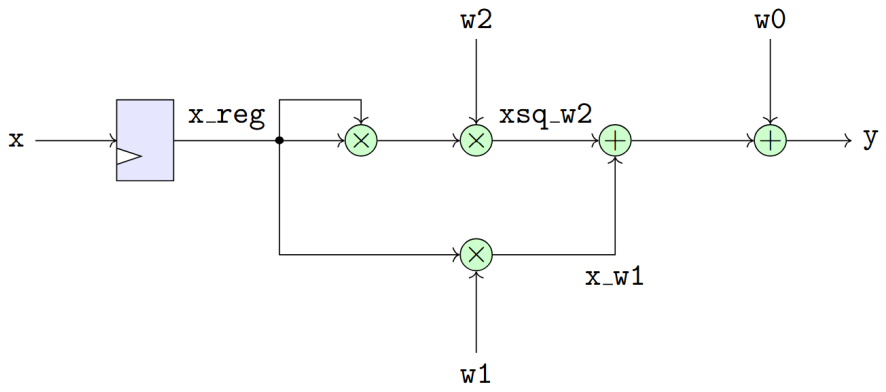
- Consider a quadratic function: $y = w_2 x^2 + w_1 x + w_0$
- For simplicity, assume w_0, w_1, w_2 are fixed parameters
- Single cycle implementation to the right




```
module quad_func #(
  parameter int w2 = 0,
  parameter int w1 = 0,
  parameter int w0 = 0
) (
  input logic clk,
  input logic int x,
  output logic int y
);
  int xreg;
  always_ff @(posedge clk) begin
    xreg <= x;
  end
  always_comb begin
    logic int xsq, xsq_w2, x_w1;
    xsq = xreg * xreg;
    xsq_w2 = w2 * xsq;
    x_w1 = w1 * xreg;
    y = xsq_w2 + x_w1 + w0;
  end
```

Finding the Critical Path

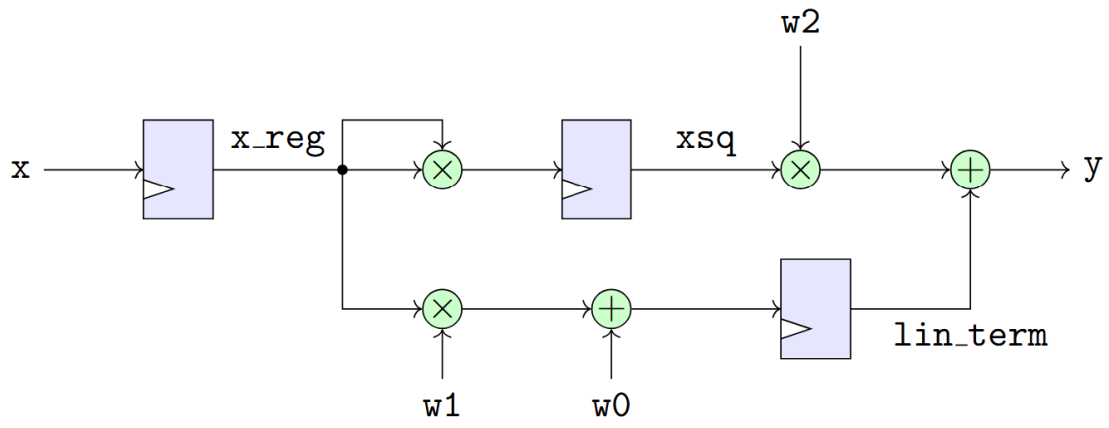
Hardware block	Example Value
Register	T_{reg}= 1 ns
Multiplier	T_{mult} = 4 ns
Adder	T_{add} =1.5 ns
Path 1	1+4+4+1.5+1.5 = 12
Path 2	1+4+1.5+1.5=8



- Consider implementation with times in table
- Two paths:
- Path 1: x, x_reg, xsq_w2, y
  - 1 register, 2 mults, 2 adders = 12 ns
- Path 2: x, x_reg, x_w1, y
  - 1 register, 1 mult, 2 adders = 8 ns
- Path 1 is critical
- Max clock frequency $= \frac{1}{12\ ns} = 83$ MHz
- Would fail timing at 100 MHz

A Two Cycle Design

- We can improve throughput by breaking computation over two cycles
- Register two intermediate terms:
  - `xsq = xreg * xreg`
  - `lin_term = w1*xreg + w0`




```
int xreg;
always_ff @(posedge clk) begin
  xsq <= xreg * xreg;
  lin_term <= w1 * xreg + w0;
  xreg <= x;
end
always_comb begin
  y = w2*xsq + lin_term;
end
```

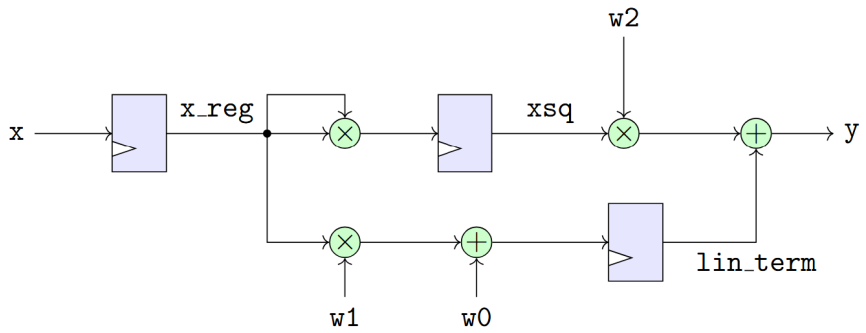
Three Stages

- Design is processed in three stages = 2 cycles
- Stage 0
  - Register input x to `xreg`
- Stage 1: Compute and register:
  - `xsq = xreg * xreg`
  - `lin_term = w1*xreg + w0`
- Stage 2: Compute output
  - `y = w2*xsq + lin_term`

```
int xreg;
always_ff @(posedge clk) begin
  xsq <= xreg * xreg;
  lin_term <= w1 * xreg + w0;
  xreg <= x;
end
always_comb begin
  y = w2*xsq + lin_term;
end
```

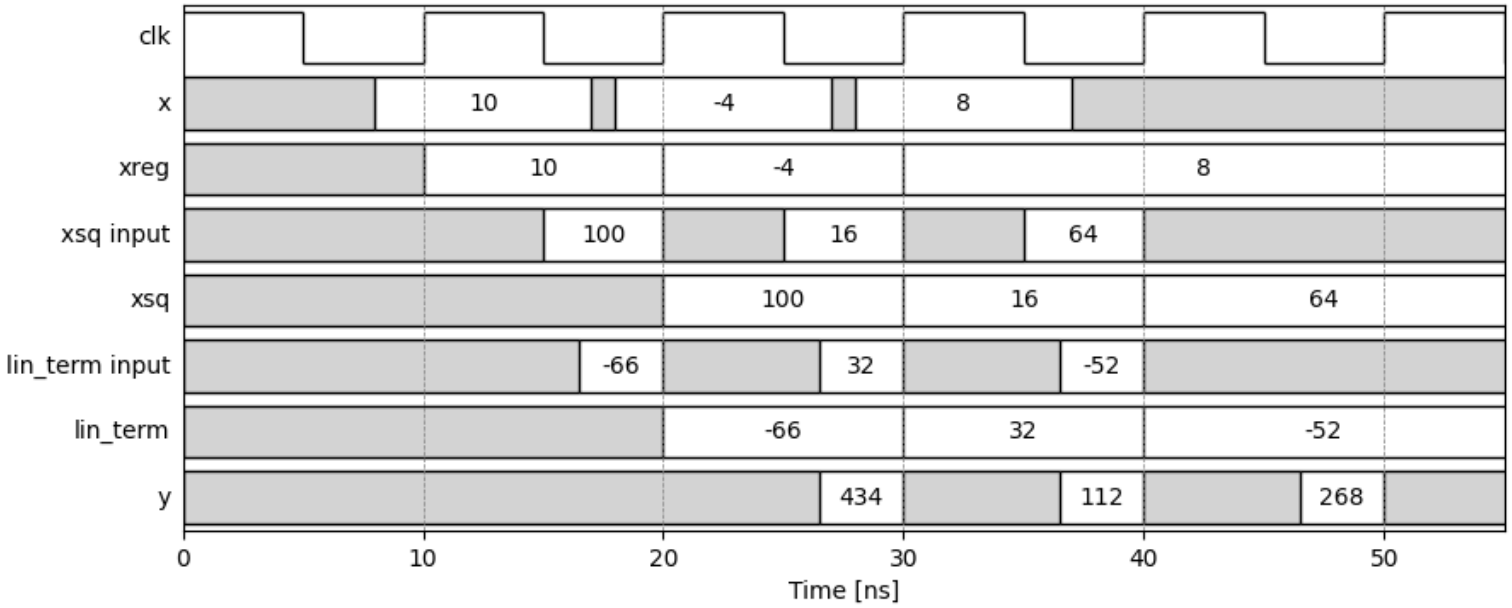
Critical Path with Three Stages

Hardware block	Example Value
Register	T_{reg}= 1 ns
Multiplier	T_{mult} = 4 ns
Adder	T_{add} =1.5 ns
Path 1	1+4=5
Path 2	1+4+1.5=6.5
Path 3	1+4+1.5=6.5



- Measure paths from register to register
- Path 1: `x, x_reg, xsq` input
  - 1 register, 1 mult = 5 ns
- Path 2: `x, x_reg, w1_x, lin_x` input
  - 1 register, 1 mult, 1 add = 6.5 ns
- Path 3 : `xsq_reg, lin_x, w2_xsq, y`
  - 1 register, 1 mult, 1 add = 6.5 ns
- Critical path delay = 6.5 ns
- Max freq = $\frac{1}{6.5} \approx 153$ MHz

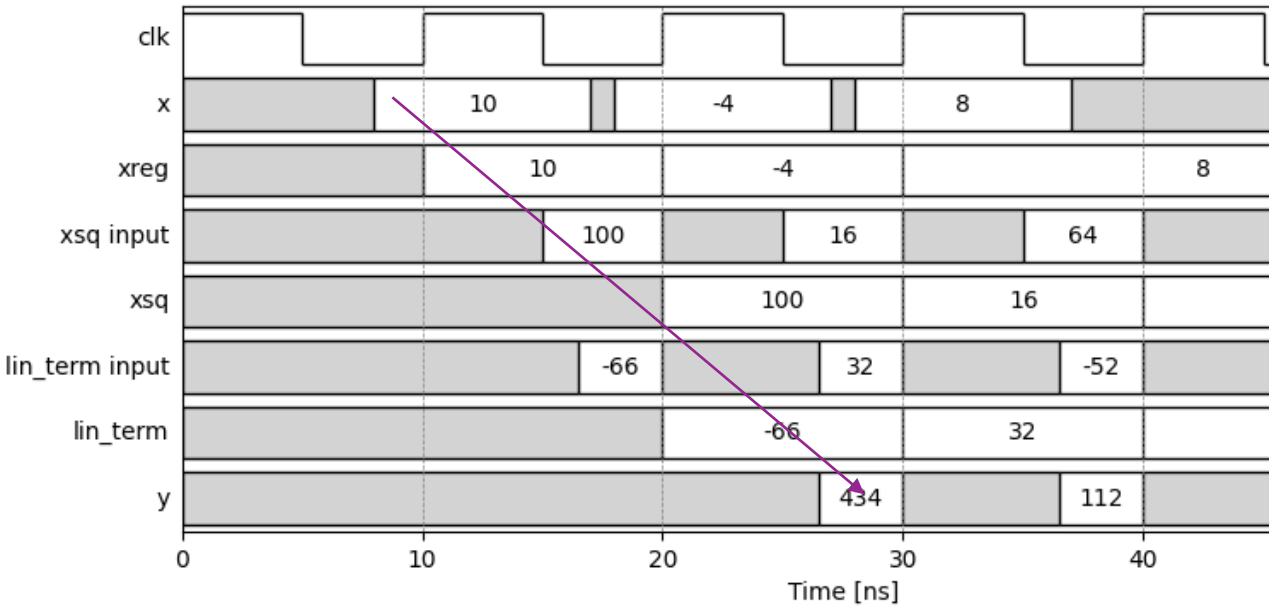
Two Cycle Timing Diagram



- Example with
  - $w = [4, -7, 5]$
- x inputs 10, -4, 8

Latency, Throughput and Pipelining

- Suppose that we run at our design at 100 MHz clock
- Latency:
  - Suppose input is valid on rising edge of clock n
  - Then output is valid before rising edge of clock $n + 2$
  - Latency = 2 clock cycles = 20 ns
- Throughput:
  - Circuit can take a new input each cycle
  - Throughput = 1 input / cycle = 100 MHz
- Process is called a pipeline
  - Circuit is processing multiple inputs at the same time



Performance Comparison

	Single Cycle Design	Two Cycle Design
Critical path	12 ns	6.5 ns
Max clock frequency	≈ 80 MHz	≈ 150 MHz
Max throughput	80 MHz	150 MHz
Latency	1 cycle = 12 ns	2 cycles = 13 ns

Pipelining benefit:
For small increase in latency, we dramatically improve throughput

Outline

- ❑ Static Timing Analysis in Combinational Logic
- ❑ Static Timing Analysis in Sequential Logic
- ❑ Sequential Logic Timing Diagrams
- ❑ VCD Files and Timing Diagrams from Vitis

Value-Change Points

- Simulation tools trace values over time for large numbers of signals
- Data on each signal is typically stored via value-change points

Definition: A signal $x(t)$ has value-change points $\{(t_i, x_i), i = 0, \dots, n-1\}$ if:

$$x(t) = \begin{cases} X & t < t_0 \\ x_i & t_i \le t < t_{i+1}, \quad i = 0, \dots, n-2 \\ x_{n-1} & t \ge t_{n-1} \end{cases}$$

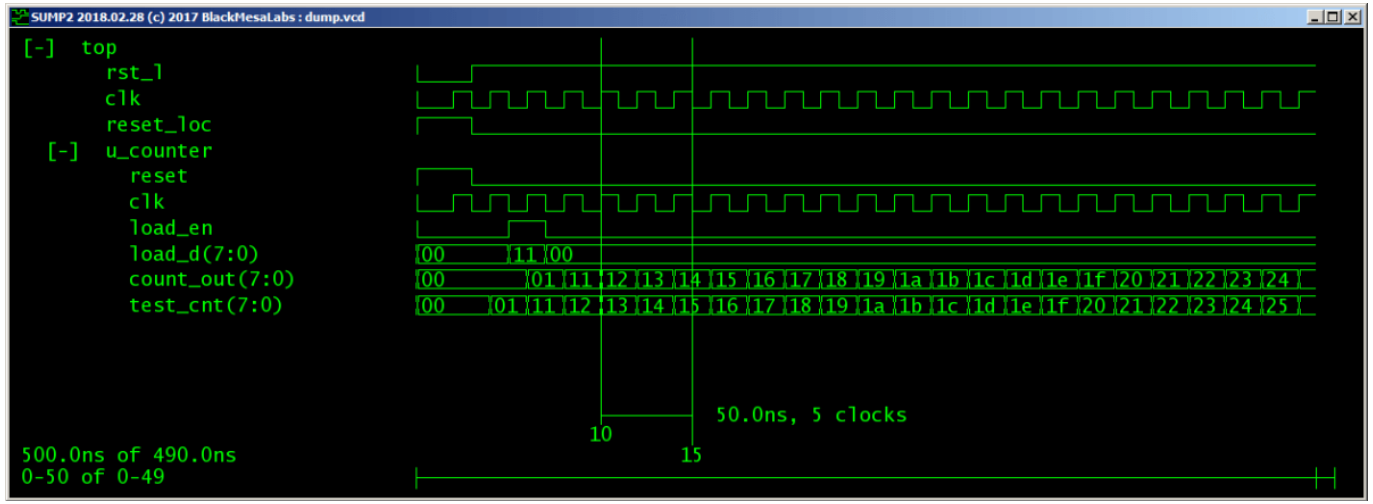
- Value change points also called valued change events, transitions, …

Timing Diagram and VCDs

- Timing diagrams can be drawn from the VCDs
- Example with two signals:
  - VC points for x : {(3,5), (20,17), (40,8)}
  - VC points for y : {(12,3), (25,12), (35,0)}
- In this plot, times are in ns

Viewing Waveforms in Vivado

- Vivado has a built-in waveform viewer
- But no way to download the data
  - Stored in a proprietary format
- Can only be seen in GUI
- As far as I know



Viewing Timing Diagrams in this Class

- Use course package xilinxutils
  - Many python utilities for Xilinx tools
- Demonstrate on simple function demo
  - Covered in Unit 2
- Sv_sim utility:
  - Modifies the vitis simulation files
  - Adds a VCD file
  - Creates a dump.vcd file
- In python:
  - Load the VCD file
  - Use VCD package

Hardware Design / Support Material / GitHub Repository / Insta

Installing `xilinxutils`

Installing the package

```
(env) sv_sim --source lin_relu.sv --tb tb_lin_relu.sv
```

```
# Parse the VCD file
import os
fn = os.path.join(os.getcwd(), "sim", "dump.vcd")
vcd = VCDVCD(fn, signals=None, store_tvs=True)
```

Parsing the Signals in Python

- ❑Create a VcdParser object
- ❑Define signals to parse
- ❑Add signals to parser

```python
# Create a parsing class
vp = VcdParser(vcd)
```

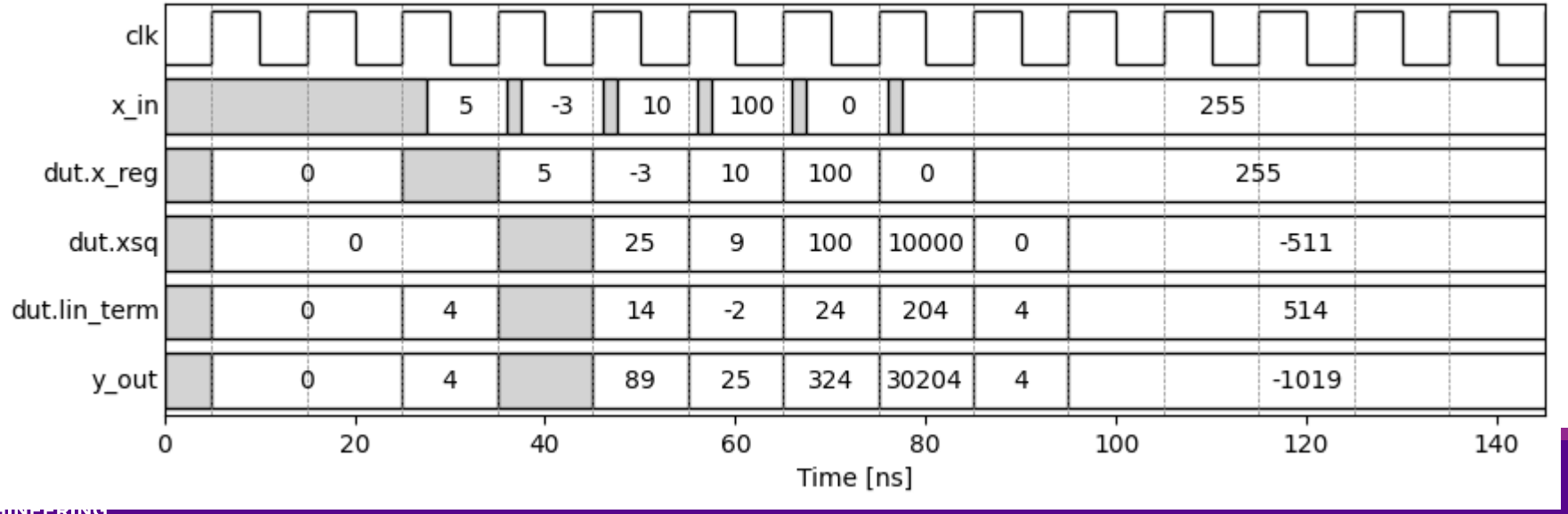
```python
signals = [
    'clk',
    'a_in',
    'b_in',
    'dut.a_reg',
    'dut.b_reg',
    'c_out'
]
```

```python
# For each signal, find it in the VCD and add it to the parser
for sig in signals:
    name = f'{prefix}.{sig}'
    found = False
    for s in vcd.signals:
        if name in s:
            vp.add_signal(s, short_name=sig,
                          numeric_type=dtype,
                          numeric_fmt_str='%d')
            print(f"Added signal: {s}")
            found = True
            break
    if not found:
        raise ValueError(f"Signal not found: {name}")
```

Displaying the Signals in Python

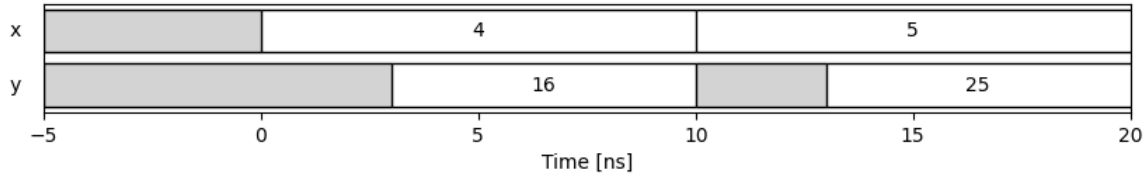
```python
# Get the timing signals
sig_list = vp.get_td_signals()

# Create the timing diagram
td = TimingDiagram()
td.add_signals(sig_list)
trange = None
ax = td.plot_signals(add_clk_grid=True, trange=trange, text_mode='always')
_ = ax.set_xlabel('Time [ns]')
```

- ❑ Create a TimingDiagram object
- ❑ Add the signals from the parser
- ❑ Plot



Drawing Timing Diagrams in Python

- Course repo contains a [xilinxutils package]
  - Custom package in this class
  - Many helper routines for Xilinx tools
- Tools for parsing and visualizing VCD files
- Code to produce timing diagram



Hardware Design / Support Material / GitHub Repository / Insta

Installing `xilinxutils`

Installing the package

```
xvals = ['x', '5', '17', '8']
yvals = ['x', '3', '12', '0']
xtimes = [0, 3, 20, 40]
ytimes = [0, 12, 25, 35]

xsig = SigTimingInfo("x", xtimes, xvals)
ysig = SigTimingInfo("y", ytimes, yvals)
td = TimingDiagram()
td.add_signals([xsig, ysig])
ax = td.plot_signals(trange=[0,50])
_ = ax.set_xlabel("Time [ns]")
```